

UNIVERSITATEA “ALEXANDRU IOAN CUZA” IAȘI
FACULTATEA DE MATEMATICĂ
SPECIALIZAREA MATEMATICĂ-INFORMATICĂ



TEHNICI DE OPTIMIZARE AI ALGORITMILOR PID

Lucrare de licență

Conducător științific:
Conf. Dr. Dănuț Rusu

Student:
Lungu Alexandru Mihai

Iunie 2022

Cuprins

Prefață	2
1 Preliminarii teoretice	3
1.1 Controlerului PID	3
1.2 Interpretarea controlerului PID	3
1.3 Tehnici de optimizare ai algoritmilor PID în cazul dronelor	4
2 Prezentarea aplicației	6
2.1 Echipamente utilizate și construcția dronei	6
2.2 Codul Sursă	11
2.2.1 ConfigurareDrona.ino	11
2.2.2 PIDdrona.ino	21
Index	30
Bibliografie	31

Prefață

Controlerele PID sunt utilizate într-o gamă largă de aplicații, inclusiv sisteme de control industrial, sisteme auto și sisteme aerospațiale. În controlul industrial, controlerele PID sunt utilizate pentru controlul temperaturii, presiunii, debitului și alte variabile de proces. În sistemele auto, acestea sunt utilizate pentru controlul vitezei de croazieră și gestionarea motorului. În industria aerospațială, ele sunt folosite pentru controlul atitudinii în sateliți și aeronave. Controlerele PID au mai multe avantaje față de alte metode de control. Ele sunt relativ simplu de proiectat și implementat, răspund rapid la schimbările variabilei de proces și nu necesită un model precis al procesului controlat. Cu toate acestea, ele au, de asemenea, unele dezavantaje, cum ar fi dificultatea de a face față sistemelor neliniare și posibilitatea de instabilitate dacă nu sunt reglate corespunzător.

Lucrarea, Tehnici de optimizare ai algoritmilor PID, constă în construcția unei drone, controlată de un microcontroler, pe care se vor implementa și experimenta diverși algoritmi pentru optimizarea parametrilor PID, astfel încât, drona să se stabilizeze în timpul zborului. Parametrizarea acestora se va face experimental, folosindu-se diverse valori reprezentative pentru acești algoritmi.

În primul capitol al acestei lucrări, se vor descrie conceptele de bază, ce intră în alcăturirea algoritmilor PID și modul în care acestea impactează construcția și proiectarea dronelor. Astfel, vom vorbi despre cele trei componente constitutive, partea de proporționalitate, integrare și derivare, pe care le vom prezenta prin diferite metode. Pe lângă acestea, datele teoretice vor fi ilustrate și pentru controlul și optimizarea zborului dronelor.

În cel de-al doilea capitol al acestei lucrări, vom prezenta construcția efectivă a dronei și vom descrie, cât se poate de amănunțit, algoritmi și conceptele folosite.

Capitolul 1

Preliminarii teoretice

1.1 Controlerului PID

Controlerul PID este un sistem de control, automat, optimizat și precis, utilizat frecvent în aplicații de control industrial, folosit pentru a regla la o valoare dorită, diferiți parametri, precum temperatura, presiunea, viteza etc. Acest mecanism funcționează pe baza "feedback-ului" sau a reacției produse de o buclă de control, care calculează, în mod repetat, diferența dintre o valoare ideală și una reală. Această diferență, apare sub denumirea de *eroare* și stabilește, modul în care, parametri procesului vor fi setați. Calculul este continuu executat, până când, controlerul este oprit.

După cum sugerează numele, controlerul PID este alcătuit din trei componente fundamentale, denumite astfel: proporționalitate (P), integrabilitate (I) și derivabilitate (D). Într-o formă ideală, valoarea de ieșire a controlerului este reprezentată ca suma celor trei componente, astfel:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t) \quad (1.1)$$

În ecuația de mai sus (1), $e(t)$ este valoarea *erorii* semnalului t , obținută din diferența valorii de referință (valoarea ideală) și cea de ieșire (valoarea reală). Coeficienții K_p , K_i și K_d reprezintă valorile parametrilor PID, ce vor fi modificați în funcție de necesitățile controlerului. Aceștia sunt independenți, unul față de celălalt, iar importanța și utilitatea fiecăruia, va fi ilustrată în capitolele ce urmează.

1.2 Interpretarea controlerului PID

Acum, să considerăm în ecuația descrisă anterior (1), pe rând, fiecare parametru egal cu o valoare diferită de 0, iar ceilalți doi, cu o valoare egală cu 0. Astfel, pentru $K_i = K_d = 0$, ecuația devine:

$$u(t) = K_p e(t) \quad (1.2)$$

Prin eliminarea celor doi coeficienți, controlerul devine unul pur proporțional. Acest fapt indică că, în orice moment de timp, controlerul este proporțional cu valoarea *erorii*, iar mărimea semnalului de ieșire va crește odată cu aceasta. Un mod pragmatic de a vizualiza acest controler este următorul: când suntem foarte aproape de valoare dorită, adică eroarea este nulă sau aproape nulă, controlerul nu va reacționa, iar în momentul în care, sistemul se depărtează de valoarea dorită, controlerul nu va face nimic pentru a se întoarce. Acesta va oscila continuu în jurul punctului. În consecință, cu cât ne înstrăinăm mai tare de ideal, pe atât de greu va fi să-l atingem. Acest fenomen poartă denumirea de starea de echilibru a erorii. În rezolvarea acestuia vine componenta de integrabilitate.

În continuare, să presupunem $K_p = K_d = 0$. În acest fel, ecuația devine:

$$u(t) = K_i \int_0^t e(\tau) d\tau \quad (1.3)$$

Funcția principală a acțiunii de integrare este de a ne asigura că valoare de ieșire a controlerului nu rămâne blocată în starea de echilibru descrisă anterior. Pentru a face posibil acest fapt, orice valoare pozitivă a erorii, oricât de mică, ne va îndemna să creștem valoarea semnalului de control. În paralel, o eroare negativă va descrește valoarea semnalului. Putem descrie această componentă ca fiind o acumulare a valorilor erorilor din trecut. În schimb, acest termen de integrabilitate tinde să încetinească controlerul, prin adunări neîncetate. Pentru a grăbi procesul, ne folosim componenta de derivabilitate.

În cele din urmă, presupunem, $K_p = K_i = 0$. Astfel, ecuația devine:

$$u(t) = K_d \frac{d}{dt} e(t) \quad (1.4)$$

Controlerul diferențial este bazat pe rata sau viteza de schimbare a erorii. Schimbarea, oferă informații despre evoluția valorii, într-un anumit moment de timp. Dinamica procesului nu permite vizualizarea modului în care variabila de control, influențează valoarea de ieșire, fapt ce cauzează întârzieri în procesul de corectare al erorii. Astfel, prin adăugarea acestui controler, putem anticipa cum ar trebui să modificăm semnalul, astfel încât, să ne aflăm mai aproape de parametrii doriți. Putem vizualiza această controler ca o funcție ce prezice viitoarele erori.

1.3 Tehnici de optimizare ai algoritmilor PID în cazul dronelor

După cum s-a văzut în subcapitolele anterioare, algoritmi PID pot fi utilizați și adaptați pentru o gamă foarte largă de aplicații. Una dintre cele mai întâlnite, este prezentă în domeniul aparatelor de zbor, mai precis, în cel al dronelor. Prin urmare, în această secțiune, se va prezenta rolul pe care, controlerul PID îl are în buna funcționare a dronelor și modul în care putem optimiza parametrii, pentru cele mai bune performanțe.

Programul încărcat pe microcontrolerul aeronavei, citește și interpretează date provenite de la un senzor atașat, urmând să calculeze viteza ideală de rotație a motoarelor,

pentru ca drona să funcționeze după preferință. Aici, obiectivul controlerului PID este de a corecta *eroarea* obținută din diferența valorii măsurate de un senzor (modulul giroscop și accelerometru) și o valoare ideală. Această *eroare* poate fi minimizată prin ajustarea corespunzătoare a parametrilor PID, folosindu-se tehnici specifice. În cele ce urmează, se va evidenția, care este efectul produs de fiecare parametru în controlul unei drone.

Termenul proporțional (P) ajută drona să răspundă rapid la schimbările din mediul său. Proporționalitatea controlerului determină sensibilitatea și capacitatea de reacție a aparatului de zbor. Dacă coeficientul K_p este prea mare, drona devine foarte sensibilă și instabilă, cu tendința de a corecta *eroarea* cu valori foarte mari, fapt ce va declanșa oscilații puternice. Cu cât valoarea lui K_p este mai mică, cu atât frecvența oscilațiilor scade, dar mișcările dronei pot deveni "dezordonate". Parametrul singur, nu este suficient pentru a stabili drona, în special pe termen lung, deoarece nu va putea elimina eroarea cauzată de perturbări externe. Acesta este motivul pentru care controlerul PID utilizează și ceilalți parametri.

Termenul de integrabilitate (I) ajută la eliminarea *erorii* ce apare în starea de echilibru, acumulând-o, pe durata zborului. Integrabilitatea controlerului poate fi indicată prin modul în care drona reacționează, când este perturbată de forțe externe, cum ar fi vântul. În general, alegerea întâmplătoare a unei valori pentru K_i va funcționa pe majoritatea dronelor. Totuși, dacă parametrul este prea mic, există posibilitatea ca drona să trebuiască corectată suplimentar în timpul zborului, prin telecomandă. În schimb, o valoare mult prea mare, poate induce, asemenea parametrului K_p , o oscilație de frecvență mai mică.

Termenul de derivabilitate (D) prezice cât de repede se schimbă *eroarea* și ajustează ieșirea controlerului în consecință, ajutând drona să se oprească în poziția dorită. Derivabilitatea controlerului are scopul de a reduce și amortiza efectul produs de corecțiile excesive pe care drona le face, din cauza parametrilor K_p și K_i . Când parametrul K_d este prea mic, drona va fi lentă și nu se va redresa la timp, dacă este mișcată sau perturbată. Problema poate fi rezolvată prin creșterea parametrului. Totuși, creșterea excesivă poate cauza vibrații și supraîncălzirea motoarelor.

Fiecare termen are un rol foarte important și specific, iar valorile acestora trebuie ajustate, astfel încât, performanțele dronei să fie cât de bune posibil. Optimizarea parametrilor PID nu este adeseori o muncă ușoară și depinde foarte mult de modelul dronei, de spațiu și de dorințele fiecăruia. Sunt foarte multe metode ce pot fi folosite, precum: modele matematice, algoritmi de inteligență artificială, tunare manuală etc. În această lucrare, optimizarea parametrilor se face manual, căutând cele mai bune raporturi dintre K_p și K_d , apoi dintre K_p și K_i , în cele din urmă maximizându-i pe toți. În funcție de reacțiile dronei cu diferite valori, analizate mai sus, se încearcă alegerea celor mai bune. În procesul acesta, sunt foarte mulți factori ce trebuie luați în considerare, precum: vibrația motoarelor, centru de greutate al dronei (ar trebui să fie chiar în mijlocul dronei, acolo unde cele patru motoare se intersectează pe un plan orizontal), distribuția greutății (dronile cu masă mai centralizată au tendința de a se simți mai precise, mai rapide și mai sensibile la comenzi.) și așa mai departe. În continuare, este prezentată aplicația, cu codurile sursă folosite, unde parametrii sunt setați cu valori optime.

Capitolul 2

Prezentarea aplicației

În acest capitol, se vor prezenta etapele parcurse pentru construcția dronei. Astfel, se vor detalia informații legate de componente, asamblare și codurile sursă folosite.

2.1 Echipamente utilizate și construcția dronei

Echipamente utilizate

Pentru a asigura comanda și controlul dronei este necesară folosirea și implementarea unor echipamente specifice, precum: un cadru de dronă cu o placă de distribuție atașată (PDB), 4 esc-uri (Electronic Speed Controller), 4 motoare, elicele, un microcontroler (Arduino UNO), un modul giroscop și accelerometru (MPU-6050), un receiver (FS-IA6B) și o telecomandă (FlySky FS-i6x). Adicional, au mai fost folosite și alte componente auxiliare, precum: barete de pini, cabluri, LED-uri, o placă de expansiune pentru Arduino UNO și o baterie LiPO de 2200 mAh cu 3 celule (3S). În continuare, vom prezenta câteva dintre particularitățile componentelor alese și importanța acestora.

Modulul giroscop și accelerometru este de tipul MPU6050. Acesta este un sistem micro-electro-mecanic (MEMS) care conține un accelerometru cu 3 axe și un giroscop cu 3 axe în interior. Componenta măsoară accelerația, viteza, orientarea, deplasarea și mulți alți parametri legați de mișcarea unui sistem sau obiect. Acest modul are, de asemenea, un procesor digital de mișcare (DMP) în interior, care este suficient de puternic pentru a efectua calcule complexe.

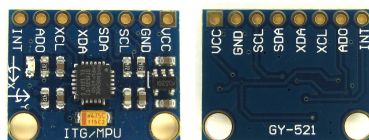


Figura 2.1: Modulul giroscop și accelerometru

Microcontroler-ul dronei este de tipul Arduino Uno R3. Această placă de dezvoltare este bazată pe microcontroler-ul ATmega328, ce conține 14 pini digitali de tipul intrare/ieșire și 6 intrări analog. Această componentă este responsabilă pentru controlul, calculul și buna funcționare a tuturor proceselor ce vor avea loc.

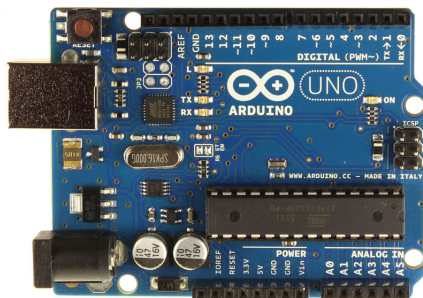


Figura 2.2: Microcontroler-ul Arduino Uno

Transmițătorul radio (FS-IA6B) este un dispozitiv electronic care utilizează semnale radio pentru a transmite comenzi, fără fir, printr-o frecvență radio setată de la receptorul radio (FlySky FS-i6x). Cu alte cuvinte, acesta este dispozitivul care traduce comenzile pilotului în mișcarea dronei.



Figura 2.3: Telecomandă FlySky FS-i6x/Receiver FS-IA6B

Termenul ESC (Electronic Speed Controller) reprezintă controlul electronic al vitezei și este un circuit electronic utilizat pentru schimbarea vitezei unui motor electric, traseului său și, de asemenea, funcționează ca o frână dinamică. Acestea sunt frecvent utilizate pe modele controlate radio, alimentate electric și pentru motoarele fără perie.

Motoarele folosite sunt fără perie. Acestea se vor învârti cu o viteză egală cu frecvența semnalului primit de la esc-uri. Ele vor fi responsabile pentru generarea puterii, cu care,

drona se va ridica in aer. Prin urmare, un pas important în construcția dronei, va fi echilibrarea motoarelor si a elicelor, astfel încât, vibrația produsă în timpul zborului să fie minimă.



Figura 2.4: Controler electronic de viteză/Motor

Cadrul este scheletul principal. Fără cadru este imposibil să creezi o dronă. Este partea cea mai vitală și asigură cele mai importante caracteristici de performanță, precum greutatea sau aerodinamica. O altă caracteristică importantă ține de design, care variază în trei forme: cadrul H, cadrul X și Stretch. Pentru acest proiect, a fost folosit unul în formă de X (F450), cu dimensiune de: 21.2 cm x 4.0 cm x 5.2 cm și o greutate de 375 g.



Figura 2.5: Cadrul Dronei F450

Construcția dronei

Acest paragraf are ca scop, prezentare succintă a modului în care este asamblată drona, din punct de vedere hardware. Astfel, pentru o înțelegere mai bună a structurii proiectului, se vor ilustra câțiva pași simpli, corelați cu imagini reprezentative și o schemă electrică.

Se începe prin a asambla cadrul dronei și a monta cele patru motoare. Apoi, se lipește firele esc-urilor de placa de distribuție (PDB) și se strâng cu coliere de brațele dronei.

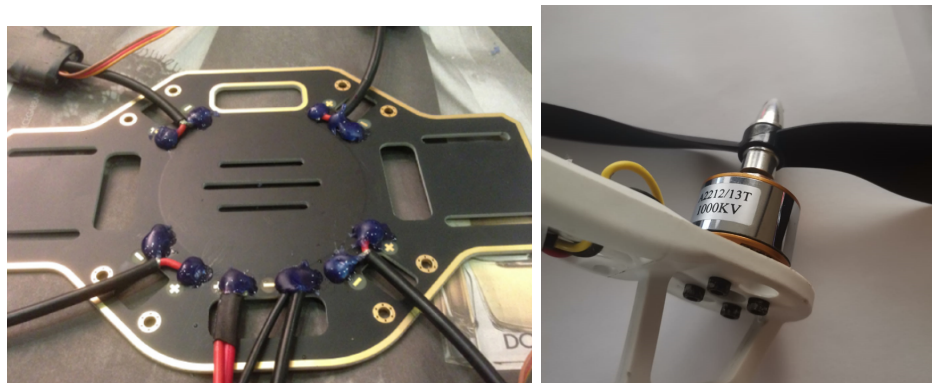


Figura 2.6: Placă de distribuție/Motor cu elice

Pentru fiecare esc, se lipește firul din mijloc al motorului corespunzător la firul din mijloc al esc-ului respectiv. Sunt conectate provizoriu firele rămase și se verifică dacă motorul se rotește corect. În caz afirmativ, firele se lipește, iar în caz negativ, se inversează conexiunile. Pentru izolare se folosește tub termocontractibil. Toate firele sunt pilate și prinse strâns cu coliere. În aceeași manieră, este fixat de cadru și receiverul RC.



Figura 2.7: ESC fixat cu coliere/Receiver fixat cu coliere

În continuare, se fixează cu coliere în centrul dronei, pe o bucată de polistiren, microcontroler-ul, Arduino UNO, împreună cu o placă de expansiune. Central, peste

placa de expansiune, este lipit modul giroscop și accelerometru, astfel încât vibrația creată de motoare să nu-l afecteze. În plus, acest mod de asamblare reduce încărcătura creată de conexiunile prin cabluri.

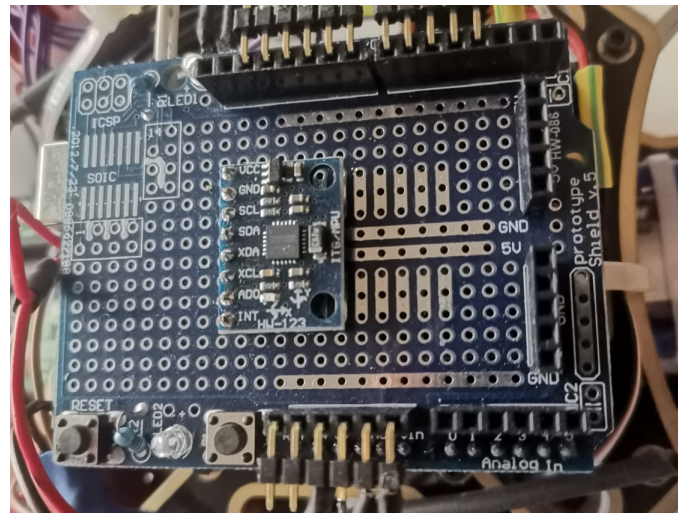


Figura 2.8: Modul giroscop și accelerometru, centrat pe placa de expansiune

Ultimul pas este reprezentat de crearea conexiunilor între componentele dronei, urmând schema electrică atașată mai jos:

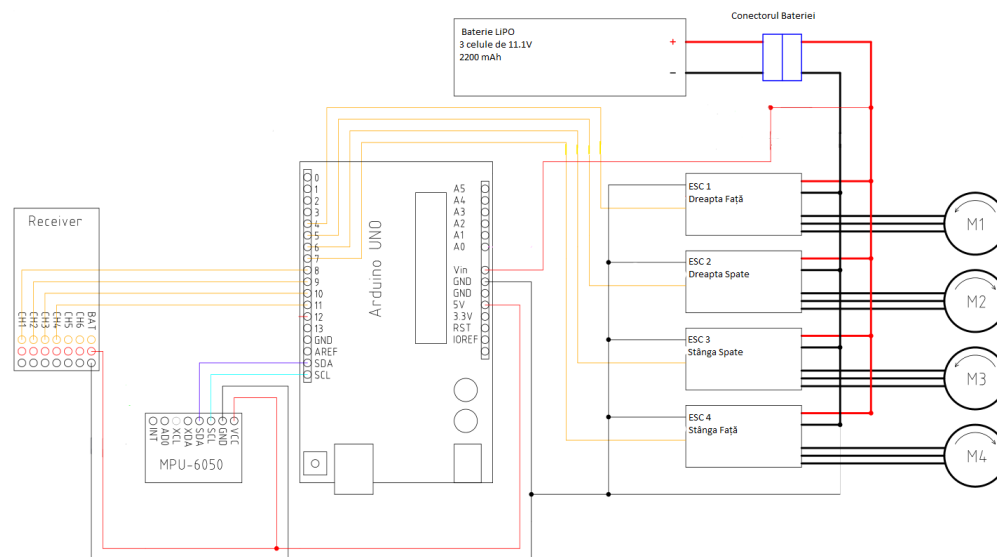


Figura 2.9: Schemă electrică

Aici, se începe cu modulul giroscop și accelerometru. Se lipesc firele pe pinii pentru alimentare (VCC, GND), datele seriale (SDA) și ceasul giroscopului (SCL). Pinii pentru SDA și SCL sunt conectați corespunzător la cei de la Arduino UNO. Mai departe, se

conectează, de la reciver-ul RC, canalele 1, 2, 3, 4 la pinii microcontroler-ului 8, 9, 10, 11. În aceeași manieră se procedează și pentru esc-uri. Pentru esc-ul 1, corespunzător motorului 1 de pe brațul drept din față, lipim cablul din mijloc, la pinul numărul 4 al microcontroler-ului. La fel procedăm și pentru esc-urile 2, 3, 4 la pinii 5, 6, 7. Într-un final, conectăm alimentarea, de la bateria LiPo, la placa de distribuție (PDB) și la pinul VIN, de pe Arduino.

2.2 Codul Sursă

În acest subcapitol se vor prezenta codurile sursă folosite și rolul acestora. Extensia fișierelor ce conțin codurile, specifică microcontroler-ului Arduino Uno, este ".ino". Pe drona, vor fi încărcate, pe rând, două fișiere sursă: ConfigurareDrona.ino și PIDdrona.ino, care vor fi prezentate în paragrafele următoare. Dar, înainte de acestea, se vor prezenta în câteva rânduri, capacitățile și limitele pe care acestă aparat de zbor le are.

Dronele, prin natura lor sunt foarte instabile. Stabilitatea acestora este obținută prin procesarea unui enorm număr de corecții pe secundă. Pentru acest proiect, drona este capabilă să performeze 250 de corecții pe secundă, proces ce apare sub denumirea de "refresh rate" sau rată de îmborsărire. Astfel, esc-urile vor fi corectate la fiecare 4 milisecunde. Din cauza faptului că, rata de îmborsărire a aparatului de zbor este diferită de cea a reciver-ului, sincronizarea celor doua este imposibilă. Acest fapt, ne sugerează folosirea unei proprietăți puternice, pe care procesorul microcontroler-ului o deține, anume, posibilitatea de a declanșa întreruperi pe pini. Un alt aspect important, ce trebuie menționat în acest paragraf, este legat de datele de ieșire ale modulului giroscop și accelerometru. În manualului de utilizare, modulul este descris, ca fiind un senzor ce receptează mișcările unghiulare, pe cele 3 axe. Deoarece, acesta simte doar rotația în jurul axelor, datele de ieșire nu vor fi influențate, dacă mișcăm drona fără a o roti. Datorită acestei proprietăți, acest senzor devine ideal, pentru drona noastră. Detaliile tehnice, pentru cele menționate mai sus, vor fi acoperite în cele ce urmează.

2.2.1 ConfigurareDrona.ino

După cum sugerează numele, fișierul ConfigurareDrona.ino, are scopul de a structura, testa și configura variabile referitoare la semnale, măsurători și diferite date semnificative, provenite de la componentele ce intra în alcătuirea dronei. După ce este încărcat pe microcontroler, programul facilitează comunicare dintre utilizator și drona, specificând pașii ce trebuiesc urmați pentru o configurare corectă.

Programul folosește două librării importante, Wire.h și EEPROM.h. Wire.h este o librărie ce facilitează comunicarea dintre circuitele integrate, mai precis, dintre modulul giroscop și microcontroler, prin utilizarea unor funcții specifice. EEPROM.h este librăria prin care se vor stoca datele necesare configurării, într-o memorie specifică, denumită EEPROM. Totodată, programul prezintă două funcții principale, setup() și loop(). Funcția,

setup() va fi apelată o singură dată și este utilizată pentru a inițializa variabile, modul pinilor, librării etc. În cazul acesta, sunt setați registrii care vor declanșa întreruperile, PCICR (pentru cele 3 porturi de pini) și PCMSK0 (pentru pinii 8, 9, 10, 11), dar și inițializarea microcontroler-ului ca fiind Master (specific protocolului I2C). După crearea funcției setup(), apare funcția loop(), care face exact ceea ce sugerează numele său, repetă blocul de cod din interiorul ei, la nesfârșit. Singurele momente în care repetiția este oprită apar când sunt declanșate întreruperi pe pinii anterior menționați, moment în care, este apelată funcția ISR(), unde sunt calculate semnalele venite de la receiver. În funcția loop(), odată cu deschiderea monitorului serial, începe comunicarea cu utilizator, prin diverse instrucțiuni printate pe monitorul serial. Configurarea va fi făcută pentru semnalele primite de la receiver, prin memorarea diferitelor poziții ale manetelor (în poziție centrală, minimă, maximă etc) dar și pentru diversele date de ieșire provenite de la modulul giroscop (ridicând drona la 45 de grade spre stânga, dreapta etc). Astfel, data pentru canalele de comunicare, semnale, adresele registrilor și multe altele sunt stocate în memoria EEPROM.

```
// includem libraria Wire.h pentru a comunica cu modulul giroscop prin protocolul I2C
#include <Wire.h>
// includem libraria EEPROM.h pentru a memora informatii in libraria EEPROM
#include <EEPROM.h>
// declararea variabilelor globale
byte last_channel_1, last_channel_2, last_channel_3, last_channel_4;
byte lowByte, highByte, tipul, adresa_giroscop, flag_eroare, clockspeed_ok;
byte canal_1_atribuit, canal_2_atribuit, canal_3_atribuit, canal_4_atribuit;
byte axa_roll, axa_pitch, axa_yaw;
byte byte_semnale_reciver, gyro_check_byte;
volatile int reciver_canal_1, reciver_canal_2, reciver_canal_3, reciver_canal_4;
int reciver_semnal_central_1, reciver_semnal_central_2, reciver_semnal_central_3, reciver_semnal_central_4;
int max_semnal_canal_1, max_semnal_canal_2, max_semnal_canal_3, max_semnal_canal_4;
int min_semnal_canal_1, min_semnal_canal_2, min_semnal_canal_3, min_semnal_canal_4;
int adresa, index;
unsigned long timp, timp_1, timp_2, timp_3, timp_4, timp_curent_intreruperi;
float giro_pitch, giro_roll, giro_yaw;
// functia de setup unde setam pinii care vor declansa intreruperi
// tot aici, se seteaza microcontroler-ul ca fiind Master, pentru protocolul I2C
void setup(){
    // registrul PCICR controleaza 3 porturi de pini care pot declansa intreruperi
    // aceste 3 porturi/grupuri sunt controlate de alti 3 registri (PCMSK0, PCMSK1, PCMSK2)
    PCICR |= B00000001; //setam PCICR pentru a activa registrul PCMSK0, care controleaza pinii 8,9,10 si 11
    PCMSK0 |= B00000001; // setam registrul PCINT0 (pinul 8) sa declanseze o intrerupere
    PCMSK0 |= B00000010; // setam registrul PCINT1 (pinul 9) sa declanseze o intrerupere
    PCMSK0 |= B00000100; // setam registrul PCINT2 (pinul 10) sa declanseze o intrerupere
    PCMSK0 |= B00010000; // setam registrul PCINT3 (pinul 11) sa declanseze o intrerupere
    Wire.begin(); // Incepem comunicarea prin I2C, cu microcontroler-ul ca fiind Master
    Serial.begin(57600); // incepem conexiunea cu minitorul serial @ 57600bps
    delay(400); // dam timp giroscopului si reciverului sa porneasca
}
// incepem programul principal, o functie care se va tot repeta
// aici vor avea loc testele si memorarea variabilelor necesare
void loop(){
    Serial.println(F("Incepe Testarea sistemului"));
    Serial.println(F("Se verifica daca clock-ul din protocolul I2C este setat la viteza corespunzatoare"));
    delay(1000);
    // setam viteza de comunicare a I2C-ului la 400 kHz (modul rapid)
    // ca setare initiala, aceasta viteza este de 100kHz (modul minim)
    Wire.setClock(400000); // analog, se poate seta si prin schimbarea registrului TWBR = 12
    Serial.print(F("Se verifica semnalele reciverului"));
    delay(1000);
    semnale_reciver(); // verificam daca exista semnal de la reciver, prin toate cele 4 canale de comunicare
```

```

if(flag_eroare == 0){ // daca nu am avut eroare anterior
    delay(1000);
    Serial.println(F("Verificam valorile, cu manetele in pozitie centrala"));
    delay(10000); // asteptam cateva secunde pentru a pune matele in pozitie centrala
    // vrem sa memoram valorile semnalelor cand manetele sunt in pozitie centrala
    // aceste valori vor fi folosite ca repere in procesul de configurare dar si in functionalitatea dronei
    verificare_pozitie_centrala();
}
// vom verifica, in particular, semnale pentru miscarile manetelor
if(flag_eroare == 0){ // daca nu am avut eroare anterior
    // verificam miscarea throttle-ului
    Serial.println(F("Se va misca maneta throttle-ului (stanga) si inapoi in centru"));
    verificare_semnale_reciver(1); // trimitem valoare 1, ca valoare de referinta
    Serial.print(F("Throttle este conectat la pinul digital: "));
    Serial.println((canal_3_tribuit & 0b00000111) + 7); // afisam pinul digital corespunzator
    // pentru a trece la urmatoarea configurare, vrem sa incepem cu manetele din pozitie centrala
    manete_pozitie_centrala(); // apelam functia care verifica semnalele pentru pozitie centrala
}
if(flag_eroare == 0){ // daca nu am avut eroare anterior
    // verificam miscarea pentru roll
    Serial.println(F("Se va misca maneta pentru roll (dreapta) spre dreapta si inapoi in centru"));
    verificare_semnale_reciver(2); // trimitem valoare 2, ca valoare de referinta
    Serial.print(F("Roll-ul este conectat la pinul digital: "));
    Serial.println((canal_1_tribuit & 0b00000111) + 7); // afisam pinul digital corespunzator
    manete_pozitie_centrala(); // apelam functia care verifica semnalele pentru pozitie centrala
}
if(flag_eroare == 0){ // daca nu am avut eroare anterior
    // verificam miscarea pentru pitch
    Serial.println(F("Se va misca maneta pentru pitch (dreapta) in jos si inapoi in centru"));
    verificare_semnale_reciver(3); // trimitem valoare 3, ca valoare de referinta
    Serial.print(F("Pitch-ul este conectat la pinul digital: "));
    Serial.println((canal_2_tribuit & 0b00000111) + 7); // afisam pinul digital corespunzator
    manete_pozitie_centrala(); // apelam functia care verifica semnalele pentru pozitie centrala
}
if(flag_eroare == 0){ // daca nu am avut eroare anterior
    // verificam miscarea pentru yaw
    Serial.println(F("Se va misca maneta pentru yaw (stanga) in dreapta si inapoi in centru"));
    verificare_semnale_reciver(4); // trimitem valoare 3, ca valoare de referinta
    Serial.print(F("Yaw-ul este conectat la pinul digital: "));
    Serial.println((canal_4_tribuit & 0b00000111) + 7); // afisam pinul digital corespunzator
    manete_pozitie_centrala(); // apelam functia care verifica semnalele pentru pozitie centrala
}
// vom verifica valoarea semnalelor in pozitiile sale extreme
if(flag_eroare == 0){ // daca nu am avut eroare anterior
    Serial.println(F("Se vor misca ambele manete, simultan, spre punctele extreme"));
    valori_min_max(); // cautam valorile minime si maxime ale canalelor receiverului.
    Serial.println(F("Valorile maxime, minime si centrale:")); // afisam valorile
    Serial.print(F("Digital input 08 values:"));
    Serial.print(min_semnal_canal_1);
    Serial.print(reciver_semnal_central_1);
    Serial.print(max_semnal_canal_1);
    Serial.print(F("Digital input 09 values:"));
    Serial.print(min_semnal_canal_2);
    Serial.print(reciver_semnal_central_2);
    Serial.print(max_semnal_canal_2);
    Serial.print(F("Digital input 10 values:"));
    Serial.print(min_semnal_canal_3);
    Serial.print(reciver_semnal_central_3);
    Serial.print(max_semnal_canal_3);
    Serial.print(F("Digital input 11 values:"));
    Serial.print(min_semnal_canal_4);
    Serial.print(reciver_semnal_central_4);
    Serial.print(max_semnal_canal_4);
    // cand ne vom opri din masurat, vom aduce manetele in pozitie centrala
    // si vom introduce valoare 'c' in monitorul serial, pentru a continua configurarea
}

```

```

    continuare_configurare();
}
if(flag_eroare == 0){ // daca nu am avut eroare anterior
    //Verificare model giroscop
    Serial.println(F("Cuatam modulul giroscop si accelerometru"));
    Serial.println(F("Cautam modelul MPU-6050 la adresa 0x68/104"));
    delay(1000);
    if(cautare_giroscop(0x68, 0x75) == 0x68){ //verificam daca giroscopul exista
        Serial.println(F("MPU-6050 a fost gasit la adresa: 0x68"));
        adresa_giroscop = 0x68; // completam cu adresa giroscopului
        tipul = 1; //completam si o variabila ce ne va fi utila ulterior
    }
    else{ // cazul in care giroscopul nu a fost gasit
        Serial.println(F("EROARE: NU A FOST GASIT MODULUL GIROSCOP SI ACCELEROMETRU"));
        flag_eroare = 1;
    }
    if(flag_eroare == 0){ // cazul in care a fost gasit
        delay(3000);
        Serial.println(F("Vom incepe Setup-ul modulului giroscop si accelerometru"));
        start_giroscop(); //functie care incepe setup-ul giroscopului
    }
}
// daca giroscopul a fost gasit, putem incepe calibrarea acestuia
if(flag_eroare == 0){ // daca nu exista eroare
    // facem cateva verificari pentru fiecare axa si memoram rezultatele
    Serial.println(F("Se ridica partea stanga a dronei la un unghi de aproximativ 45 de grade"));

    verificare_axe_giroscop(1); // se verifica miscarea
    if(flag_eroare == 0){ // daca nu exista eroare
        Serial.print(F("Unghiul detectat a fost = "));
        Serial.println(axes_roll & 0b00000011);
        //verificam cazul in care axele sunt inversate, care poate aparea din constructia giroscopului
        if(axes_roll & 0b10000000)Serial.println(F("Axele sunt inversate"));
        Serial.println(F("Asezati drona in pozitia initiala"));
        continuare_configurare(); // apelam functia pentru continuarea procesului
        Serial.println(F("Ridicati partea din fata dronei la un unghi de aproximativ 45 de grade"));
        verificare_axe_giroscop(2); // se verifica miscarea
    }
    if(flag_eroare == 0){ // daca nu exista eroare
        Serial.print(F("Unghiul detectat a fost = "));
        Serial.println(axes_pitch & 0b00000011);
        if(axes_pitch & 0b10000000)Serial.println(F("Axele sunt inversate"));
        Serial.println(F("Asezati drona in pozitia initiala"));
        continuare_configurare(); // apelam functia pentru continuarea procesului
        Serial.println(F("Rotiti drona in partea sa dreapta la un unghi de aproximativ 45 de grade"));
        verificare_axe_giroscop(3); // se verifica miscarea
    }
    if(flag_eroare == 0){ // daca nu exista eroare
        Serial.print(F("Unghiul detectat a fost = "));
        Serial.println(axes_yaw & 0b00000011);
        if(axes_yaw & 0b10000000)Serial.println(F("Axele sunt inversate"));
        Serial.println(F("Asezati drona in pozitia initiala"));
        continuare_configurare(); // apelam functia pentru continuarea procesului
    }
}
if(flag_eroare == 0){
    Serial.println(F("Ultima verificare:"));
    delay(1500);
    if(byte_semnale_reciver == 0b00001111){
        Serial.println(F("Semnalele receiverului sunt ok!"));
    }
    else{
        Serial.println(F("EROARE: SEMNALELE RECIVER"));
        flag_eroare = 1;
    }
}

```

```

delay(1000);
if(gyro_check_byte == 0b00000111){
    Serial.println(F("Axele giroscopului sunt ok!"));
}
else{
    Serial.println(F("EROARE: AXELE GIROSCOPULUI"));
    flag_eroare = 1;
}
}
if(flag_eroare == 0){// verificam daca exista eroare
    // retinem datele in memoria EEPROM
    Serial.println(F("Se memoreaza datele"));
    delay(2000);
    EEPROM.write(0, reciver_semnal_central_1 & 0b11111111);
    EEPROM.write(1, reciver_semnal_central_1 >> 8);
    EEPROM.write(2, reciver_semnal_central_2 & 0b11111111);
    EEPROM.write(3, reciver_semnal_central_2 >> 8);
    EEPROM.write(4, reciver_semnal_central_3 & 0b11111111);
    EEPROM.write(5, reciver_semnal_central_3 >> 8);
    EEPROM.write(6, reciver_semnal_central_4 & 0b11111111);
    EEPROM.write(7, reciver_semnal_central_4 >> 8);
    EEPROM.write(8, max_semnal_canal_1 & 0b11111111);
    EEPROM.write(9, max_semnal_canal_1 >> 8);
    EEPROM.write(10, max_semnal_canal_2 & 0b11111111);
    EEPROM.write(11, max_semnal_canal_2 >> 8);
    EEPROM.write(12, max_semnal_canal_3 & 0b11111111);
    EEPROM.write(13, max_semnal_canal_3 >> 8);
    EEPROM.write(14, max_semnal_canal_4 & 0b11111111);
    EEPROM.write(15, max_semnal_canal_4 >> 8);
    EEPROM.write(16, min_semnal_canal_1 & 0b11111111);
    EEPROM.write(17, min_semnal_canal_1 >> 8);
    EEPROM.write(18, min_semnal_canal_2 & 0b11111111);
    EEPROM.write(19, min_semnal_canal_2 >> 8);
    EEPROM.write(20, min_semnal_canal_3 & 0b11111111);
    EEPROM.write(21, min_semnal_canal_3 >> 8);
    EEPROM.write(22, min_semnal_canal_4 & 0b11111111);
    EEPROM.write(23, min_semnal_canal_4 >> 8);
    EEPROM.write(24, canal_1_atribuit);
    EEPROM.write(25, canal_2_atribuit);
    EEPROM.write(26, canal_3_atribuit);
    EEPROM.write(27, canal_4_atribuit);
    EEPROM.write(28, axa_roll);
    EEPROM.write(29, axa_pitch);
    EEPROM.write(30, axa_yaw);
    EEPROM.write(31, tipul);
    EEPROM.write(32, adresa_giroscop);
    Serial.println(F("Finalizare!"));
    while(1);
}
}
// FUNCTII PENTRU GIROSCOP
// cautam modulul giroscop si verificam registrul Who_am_I
// registrul who_am_i este folosit pentru a verifica identitatea giroscopului.
byte cautare_giroscop(int adresa_giroscop, int who_am_i){
    // incepem o transmitere de date dintre Master(Arduino) si Slave( Giroscop)
    // adresa giroscopului este data de registrul: adresa_giroscop
    Wire.beginTransmission(adresa_giroscop);
    Wire.write(who_am_i); // adaugam in coada de bytes, byte-ul: who_am_i
    Wire.endTransmission(); // opresc transmiterea de date. Trimitem coada de bytes creata anterior
    Wire.requestFrom(adresa_giroscop, 1); // se face un cerere de un singur byte (param: quantity=1)
    timp = millis() + 100; // dam la dispozitie un timp pentru procesul buclei: while()
    // Wire.available() returneaza numarul de bytes disponibili pentru citire
    // giroscop-ul poate sa trimita mai putini bytes, decat am cerut
    // vom citi, cat timp exista cel putin un byte.
    while(Wire.available() < 1 && timp > millis()); // asteptam pana cand byte-ul este primit
}

```



```

lowByte = Wire.read(); // citim un bytes
adresa = adresa_giroscop; // completam variabila globala ce memoreaza adresa
return lowByte; // returnam adresa giroscopului, 0x68
}

void start_giroscop(){
    if(tipul == 1){ // in cazul in care ne aflam pe modelul corespunzator
        Wire.beginTransmission(adresa); // incepem comunicarea cu giroscopul
        Wire.write(0x6B); // registrul 107 (PWR_MGMT_1) permite configurarea modului de alimentare si sursa
            ceasului
        //in prima faza, giroscopul vine mod activat, sleep mode
        //acesta va trebui schimbat, pentru a putea porni giroscopul
        Wire.write(0x00); // schimbam valoarea registrului PWR_MGMT_1 la 0 pentru a porni giroscopul
        Wire.endTransmission(); // oprim transmiterea de date si trimitem registrul
        Wire.beginTransmission(adresa); // incepem comunicarea cu giroscopul
        Wire.write(0x6B); // scriem registrul PWR_MGMT_1
        Wire.endTransmission(); // oprim transmiterea de date
        Wire.requestFrom(adresa, 1); // solicitam un byte de la giroscop
        while(Wire.available() < 1); // asteptam pana cand un byte este primit
        Serial.print(F("Register 0x6B is set to:")); // verificam prin acel byte, setarea a avut loc
        Serial.println(Wire.read(),BIN); // BIN este o constanta ce ne ajuta sa printam rezultatul in baza 2
        Wire.beginTransmission(adresa); // incepem comunicarea cu giroscopul
        // registrul GYRO_CONFIG (27 sau 0x1B) este folosit pentru a declansa autotestarea giroscopului
        // astfel, se permite testare portiunilor mecanice sau electrice
        // exista 3 registri (XG_ST,YG_ST,ZG_ST) care pot oferi informatii despre axele giroscopului
        // acesti registri pot oferi date independent sau simultan
        Wire.write(0x1B); // anuntam ca vrem sa scriem in registrul GYRO_CONFIG
        Wire.write(0x08); // setam registrul cu 00001000 (1 pentru bitul FS_SEL=Full scale range)
        // astfel, vom avea un interval de +/-500 degree/second
        Wire.endTransmission(); // oprim transmiterea de date
        Wire.beginTransmission(adresa); // incepem comunicarea cu giroscopul
        Wire.write(0x1B); // vrem sa citim date de la registrul GYRO_CONFIG
        Wire.endTransmission(); // oprim transmiterea de date
        Wire.requestFrom(adresa, 1); // solicitam un byte de la giroscop
        while(Wire.available() < 1); // asteptam pana cand byte-ul este primit
        Serial.print(F("Register 0x1B is set to:"));
        Serial.println(Wire.read(),BIN); // verificam byte-ul
    }
}

// aceasta este principala functie care citeste date de la axele OX, OY, OZ ale giroscopului
// exista o proprietate de autoincrementare a registrilor, pentru a citi datele in acelasi moment de timp
// datele sunt stocate in 16 biti (2 registri de cate 8 biti)
// acesti 16 biti se obtin prin combinarea a 2 registri de 8 biti
// pentru a extrage datele complete, bitii trebuie siftati pe pozitiile corespunzatoare
void semnale_giroscop(){
    if(tipul == 1){ // daca suntem pe tipul corespunzator
        Wire.beginTransmission(adresa); // incepem comunicarea cu giroscopul
        Wire.write(0x43); // registrul Gyroscope Measurements (0x43, 67) stocheaza cele mai recente masuratori
            ale giroscopului
        Wire.endTransmission(); // oprim transmiterea de date
        Wire.requestFrom(adresa,6); // solicitam giroscopului 6 bytes
        while(Wire.available() < 6); // asteptam pana cand cei 6 bytes sunt primiti
        // pentru a avea datele complete, trebuie sa aplicam cateva operatii pe biti
        // prima citire este pentru high byte-ul care se sifteaza cu 8 bites la stanga
        // peste spatiul creat se copiaza low byte-ul
        giro_roll=Wire.read()<<8|Wire.read(); //Citim byte-ul high si cel low pentru a extrage date pentru axa OX
            a giroscopului
        giro_pitch=Wire.read()<<8|Wire.read(); //Citim byte-ul high si cel low pentru a extrage date pentru axa
            OY a giroscopului
        giro_yaw=Wire.read()<<8|Wire.read(); //Citim byte-ul high si cel low pentru a extrage date pentru axa
            OZ a giroscopului
    }
}

//verificam cum sunt primite datele pentru axele giroscopului, raportat la miscarea dronei
void verificare_axe_giroscop(byte miscare_drona){
    byte miscare_axa = 0;

```

```

float giro_roll_unghi, giro_pitch_unghi, giro_yaw_unghi; // initializam variabile pentru unghiurile
corespunzatoare
//initializam valorile cu 0
giro_roll_unghi = 0;
giro_pitch_unghi = 0;
giro_yaw_unghi = 0;
//apelam functia pentru a cere date de la giroscop
semnale_giroscop();
timp = millis() + 10000; //punem o limita de timp pentru procesul buclei: while()
// acest bloc de cod ruleaza pana cand datele unghiulare sunt schimbate sau cand timpul expira
// este de preferat ca miscarile sa corespunda cu cerinta din mesajul anterior
while(timp > millis() && giro_roll_unghi > -30 && giro_roll_unghi < 30 && giro_pitch_unghi > -30
&& giro_pitch_unghi < 30 && giro_yaw_unghi > -30 && giro_yaw_unghi < 30){
    semnale_giroscop(); // se fac apeluri pentru a prelua date
    //LSB= least significant bit unit to the related number of radians per second
    // acum trebuie sa calculam unghiul parcurs, pentru o fiecare miscare a dronei
    // viteza de calcul este de 250(Hz) = 0.004(s)
    // conform manualului de utilizare a modulului giroscop, pentru a putea folosi date primite de la
    registri
    // vor trebui facute cateva conversii
    // pentru ca am configurat giroscopul la 500dps, avem o sensibilitate de 65.5 LBS
    // LSB reprezinta cel mai semnificativ bite reprezentativ pentru numarul de grade pe secunda
    // folosind formula din manualul de utilizare, giro_output * 1/65.5 * 0.004 = giro_output * 0.0000611
    // astfel se obtine viteza unghiulara care este adunata in variabila ce retine dinstanta parcursa in
    functie de viteza
    giro_roll_unghi += giro_roll * 0.0000611;
    giro_pitch_unghi += giro_pitch * 0.0000611;
    giro_yaw_unghi += giro_yaw * 0.0000611;
    delayMicroseconds(37000); // oferim un timp pentru ridicarea dronei
}
// dupa ce a fost simtita o miscare peste valoarea limita, se cauta axa corespunzatoare
// atribuim axei miscate functia corespunzatoare (roll, pitch sau yaw), in functie de datele unghiulare
if((giro_roll_unghi < -30 || giro_roll_unghi > 30) && giro_pitch_unghi > -30
&& giro_pitch_unghi < 30 && giro_yaw_unghi > -30 && giro_yaw_unghi < 30){
    gyro_check_byte |= 0b00000001;
    if(giro_roll_unghi < 0)miscare_axa = 0b10000001;
    else miscare_axa = 0b00000001;
}
if((giro_pitch_unghi < -30 || giro_pitch_unghi > 30) && giro_roll_unghi > -30
&& giro_roll_unghi < 30 && giro_yaw_unghi > -30 && giro_yaw_unghi < 30){
    gyro_check_byte |= 0b00000010;
    if(giro_pitch_unghi < 0)miscare_axa = 0b10000010;
    else miscare_axa = 0b00000010;
}
if((giro_yaw_unghi < -30 || giro_yaw_unghi > 30) && giro_roll_unghi > -30
&& giro_roll_unghi < 30 && giro_pitch_unghi > -30 && giro_pitch_unghi < 30){
    gyro_check_byte |= 0b00000100;
    if(giro_yaw_unghi < 0)miscare_axa = 0b10000011;
    else miscare_axa = 0b00000011;
}
if(miscare_axa == 0){ //caz in care nu a fost detectata miscarea pe o axa
    flag_eroare = 1;
    Serial.println(F("EROARE: NU A FOST DETECTATA MISCAREA"));
}
else{ //daca a fost detectata miscare, in functie memoram axa corespunzatoare
    if(miscare_drona == 1)axa_roll = miscare_axa;
    if(miscare_drona == 2)axa_pitch = miscare_axa;
    if(miscare_drona == 3)axa_yaw = miscare_axa;
}
}
// FUNCTII PENTRU SEMNALELE RECIVER-ULUI
// verificam daca valoarea de intrare se schimba intr-un anumit timp
void verificare_semnale_reciver(byte miscare_maneta){
    byte valoare_semnal = 0; // variabila care ne ajuta sa retinem canalul de comunicare
    bool primit_semnal = false; // variabila care memoreaza daca am primit semnal
}

```

```

int pulsatie_semnal_reciver; // variabila care memoreaza valoarea semnalului
timp = millis() + 30000; // adaugam un timp pentru executie
while(timp > millis() && primit_semnal == false){ //cat timp nu am primit semnal
    delay(250);
    // valorile 1750 si 1250 reprezinta intervalul in care manetele sunt in pozitie centrala
    // in momentul in care, semnalele depasesc aceste valori, stim ca a avut loc un procesul
    // in functie de caz, retinem ce canal a receptat semnalul
    if(reciver_canal_1 > 1750 || reciver_canal_1 < 1250){
        valoare_semnal = 1;
        primit_semnal = true;
        byte_semnale_reciver |= 0b00000001;
        pulsatie_semnal_reciver = reciver_canal_1;
    }
    if(reciver_canal_2 > 1750 || reciver_canal_2 < 1250){
        valoare_semnal = 2;
        primit_semnal = true;
        byte_semnale_reciver |= 0b00000010;
        pulsatie_semnal_reciver = reciver_canal_2;
    }
    if(reciver_canal_3 > 1750 || reciver_canal_3 < 1250){
        valoare_semnal = 3;
        primit_semnal = true;
        byte_semnale_reciver |= 0b00000100;
        pulsatie_semnal_reciver = reciver_canal_3;
    }
    if(reciver_canal_4 > 1750 || reciver_canal_4 < 1250){
        valoare_semnal = 4;
        primit_semnal = true;
        byte_semnale_reciver |= 0b00001000;
        pulsatie_semnal_reciver = reciver_canal_4;
    }
}
if(primit_semnal == false){ // verificam daca au fost detectate semnale
    flag_eroare = 1;
    Serial.println(F("EROARE: NU AU FOST DETECTATE SEMNALE DE LA RECIVER"));
}
// in cazul in care a fost detectat semnal
// atribuim, in functie de preferinta, miscarea pe canalul de comunicare
else{
    if(miscare_maneta == 1){
        canal_3_atribuit = valoare_semnal;
        if(pulsatie_semnal_reciver < 1250)canal_3_atribuit += 0b10000000;
    }
    if(miscare_maneta == 2){
        canal_1_atribuit = valoare_semnal;
        if(pulsatie_semnal_reciver < 1250)canal_1_atribuit += 0b10000000;
    }
    if(miscare_maneta == 3){
        canal_2_atribuit = valoare_semnal;
        if(pulsatie_semnal_reciver < 1250)canal_2_atribuit += 0b10000000;
    }
    if(miscare_maneta == 4){
        canal_4_atribuit = valoare_semnal;
        if(pulsatie_semnal_reciver < 1250)canal_4_atribuit += 0b10000000;
    }
}
}
// aceasta functie este folosita pentru a ne ajuta in pasii de configurare
void continuare_configurare(){
    Serial.print(F("Introduceti de la tastatura litera 'c' pentru a continua configurarea:"));
    byte data;
    if(Serial.available() > 0){
        while(data != 'c'){
            data = Serial.read(); //citeste valoarea pentru a continua.
            if(data == 'c')

```

```

        manete_pozitie_centrala();
    }
} else {
    Serial.print(F("EROARE: Monitorul serial nu este deschis"));
}
}

// verificam daca manetele sunt in pozitie centrala
void manete_pozitie_centrala(){
    byte zero = 0; // byte-ul care memoreaza daca sunt in pozitie centrala
    const int tol = 20; // toleranta, in cazul in care manetele sunt cu aproximatie, in pozitie centrala
    while(zero < 15){ // 15 = 0b00001111;
        if(reciver_canal_1 < reciver_semnal_central_1 + tol && reciver_canal_1 > reciver_semnal_central_1 - tol)
            zero |= 0b00000001;
        if(reciver_canal_2 < reciver_semnal_central_2 + tol && reciver_canal_2 > reciver_semnal_central_2 - tol)
            zero |= 0b00000010;
        if(reciver_canal_3 < reciver_semnal_central_3 + tol && reciver_canal_3 > reciver_semnal_central_3 - tol)
            zero |= 0b00000100;
        if(reciver_canal_4 < reciver_semnal_central_4 + tol && reciver_canal_4 > reciver_semnal_central_4 - tol)
            zero |= 0b00001000;
        delay(100);
    }
}

// functie care verifica existenta semnalelor primite de la reciver
// variabilele sunt actualizate mereu, prin functia care trateaza intreruperile (ISR())
void semnale_reciver(){
    byte byte_existentia_semnale_reciver = 0; // variabila care memoram daca exista semnal
    timp = millis() + 10000; // dam un timp buclei while, pentru a nu o repeta nesfarsit
    while(timp > millis() && byte_existentia_semnale_reciver < 15){ //15 = 0b00001111
        // valorile semnalului ar trebui sa fie cuprinse intre 900 si 2100
        // daca semnalul este valid, memoram o adresa de memorie corespunzatoare in variabila declarata mai sus
        if(reciver_canal_1 < 2100 && reciver_canal_1 > 900)byte_existentia_semnale_reciver |= 0b00000001;
        if(reciver_canal_2 < 2100 && reciver_canal_2 > 900)byte_existentia_semnale_reciver |= 0b00000010;
        if(reciver_canal_3 < 2100 && reciver_canal_3 > 900)byte_existentia_semnale_reciver |= 0b00000100;
        if(reciver_canal_4 < 2100 && reciver_canal_4 > 900)byte_existentia_semnale_reciver |= 0b00001000;
        delay(500); // intarziem procesul, pentru a oferi timp procesului
    }
    if(byte_existentia_semnale_reciver == 0){ //verificam daca exista semnale
        flag_eroare = 1; // tratam eroarea
        Serial.println(F("EROARE: NU AU FOST DETECTATE SEMNALE"));
    }
}

void verificare_pozitie_centrala(){
    // memoram in variabile semnalele pentru pozitia centrala
    reciver_semnal_central_1 = reciver_canal_1;
    reciver_semnal_central_2 = reciver_canal_2;
    reciver_semnal_central_3 = reciver_canal_3;
    reciver_semnal_central_4 = reciver_canal_4;
    // le afisam pe monitorul serial
    Serial.print(F("Pinul 8 = "));
    Serial.println(reciver_canal_1);
    Serial.print(F("Pinul 9 = "));
    Serial.println(reciver_canal_2);
    Serial.print(F("Pinul 10 = "));
    Serial.println(reciver_canal_3);
    Serial.print(F("Pinul 11 = "));
    Serial.println(reciver_canal_4);
    Serial.println(F(""));
}

// functie care memoreaza valorile maxime si minime ale pozitiei manetelor
void valori_min_max(){
    byte zero = 0; // byte folosit daca a fost trimis semnal pe un anumit canal
    const int tol = 10; // toleranta pentru pozitie centrala
    // dam cateva valori de start pentru semnalele minime
    min_semnal_canal_1 = reciver_canal_1;
    min_semnal_canal_2 = reciver_canal_2;

```

```

min_semnal_canal_3 = reciver_canal_3;
min_semnal_canal_4 = reciver_canal_4;
delay(250);
Serial.println(F("Incepem masurarea valorilor extreme...."));
while(zero < 15){ // cat timp au fost detectate toate semnalele (15 = 0b00001111)
    if(reciver_canal_1 < min_semnal_canal_1)min_semnal_canal_1 = reciver_canal_1;
    if(reciver_canal_2 < min_semnal_canal_2)min_semnal_canal_2 = reciver_canal_2;
    if(reciver_canal_3 < min_semnal_canal_3)min_semnal_canal_3 = reciver_canal_3;
    if(reciver_canal_4 < min_semnal_canal_4)min_semnal_canal_4 = reciver_canal_4;
    if(reciver_canal_1 > max_semnal_canal_1)max_semnal_canal_1 = reciver_canal_1;
    if(reciver_canal_2 > max_semnal_canal_2)max_semnal_canal_2 = reciver_canal_2;
    if(reciver_canal_3 > max_semnal_canal_3)max_semnal_canal_3 = reciver_canal_3;
    if(reciver_canal_4 > max_semnal_canal_4)max_semnal_canal_4 = reciver_canal_4;
    // conditiile de oprire a procesului de masurare
    // cand manetele sunt asezate din nou in pozitie centrala
    if(reciver_canal_1 < reciver_semnal_central_1 + tol && reciver_canal_1 > reciver_semnal_central_1 - tol)
        zero |= 0b00000001;
    if(reciver_canal_2 < reciver_semnal_central_2 + tol && reciver_canal_2 > reciver_semnal_central_2 - tol)
        zero |= 0b00000010;
    if(reciver_canal_3 < reciver_semnal_central_3 + tol && reciver_canal_3 > reciver_semnal_central_3 - tol)
        zero |= 0b00000100;
    if(reciver_canal_4 < reciver_semnal_central_4 + tol && reciver_canal_4 > reciver_semnal_central_4 - tol)
        zero |= 0b00001000;
    delay(100);
}
}
//This routine is called every time input 8, 9, 10 or 11 changed state
// aceasta functie este apelata de fiecare data cand valorile de intrare de la pinii 8,9,10 sau 11 se
// schimba
ISR(PCINT0_vect){ // rutina intreruperilor
    timp_curent_intreruperi = micros(); //memoram timpul curent al microcontroler-ului
    // PINB este registrul care controleaza portul piniilor de la 8 la 13
    // verificam de unde a fost declansata o intrerupere
    // canal 1
    if(PINB & B00000001){ // verificam daca avem intrerupere pe pinul 8
        // variabila care ne spune ca a inceput intreruperea
        if(last_channel_1 == 0){ // verificam daca nu exista intrerupere
            last_channel_1 = 1; // schimbam valoarea spunand ca avem intrerupere
            timp_1 = timp_curent_intreruperi; //setam timp_1 cu timp_curent_intreruperi
        }
    }
    else if(last_channel_1 == 1){ // verificam daca exista intrerupere
        last_channel_1 = 0; // schimbam valoarea spunand ca nu mai avem intrerupere
        reciver_canal_1 = timp_curent_intreruperi - timp_1; // valoare canalului va fi egala cu
            timp_curent_intreruperi - timp_1
    }
    // canal 2
    if(PINB & B00000010 ){ // verificam daca avem intrerupere pe pinul 9
        if(last_channel_2 == 0){ // verificam daca nu exista intrerupere
            last_channel_2 = 1; // schimbam valoarea spunand ca avem intrerupere
            timp_2 = timp_curent_intreruperi; //setam timp_2 cu timp_curent_intreruperi
        }
    }
    else if(last_channel_2 == 1){ // verificam daca exista intrerupere
        last_channel_2 = 0; // schimbam valoarea spunand ca nu mai avem intrerupere
        reciver_canal_2 = timp_curent_intreruperi - timp_2; // valoare canalului va fi egala cu
            timp_curent_intreruperi - timp_2
    }
    // canal 3
    if(PINB & B00000100 ){ // verificam daca avem intrerupere pe pinul 10
        if(last_channel_3 == 0){ // verificam daca nu exista intrerupere
            last_channel_3 = 1; // schimbam valoarea spunand ca avem intrerupere
            timp_3 = timp_curent_intreruperi; //setam timp_3 cu timp_curent_intreruperi
        }
    }
}
}

```

```

else if(last_channel_3 == 1){ // verificam daca exista intrerupere
    last_channel_3 = 0; // schimbam valoare spunand ca nu mai avem intrerupere
    reciver_canal_3 = timp_curent_intreruperi - timp_3; // valoare canalului va fi egala cu
        timp_curent_intreruperi - timp_3
}
// canal 3
if(PINB & B00001000){ // verificam daca avem intrerupere pe pinul 11
    if(last_channel_4 == 0){ // verificam daca nu exista intrerupere
        last_channel_4 = 1; // schimbam valoarea spunand ca avem intrerupere
        timp_4 = timp_curent_intreruperi; //setam timp_4 cu timp_curent_intreruperi
    }
}
else if(last_channel_4 == 1){ // verificam daca exista intrerupere
    last_channel_4 = 0; // schimbam valoare spunand ca nu mai avem intrerupere
    reciver_canal_4 = timp_curent_intreruperi - timp_4; // valoare canalului va fi egala cu
        timp_curent_intreruperi - timp_4
}
}
}

```

2.2.2 PIDdrona.ino

În acest subcapitol, este prezentat codul sursă ce este responsabil pentru buna funcționare a dronei. Aici, vor fi evidențiate conceptele importante și rolul acestora, astfel încât, aparatul de zbor să fie utilizat la capacități maxime. Astfel, pentru o mai bună înțelegere, sunt ilustrate câteva etape esențiale ce se regăsesc în codurile de mai jos.

După cum a fost prezentat și în capitolele anterioare, motoarele sunt controlate de cele patru esc-uri. La rândul lor, esc-urile sunt controlate de microcontroler-ul Arduino Uno, prin date ce sunt cuprinse între 1000 și 2000 de milisecunde. Datele acestea sunt colectate de la reciver-ul RC printr-o funcție ce rulează continuu în fundalul programului principal, anume ISR(). Aceasta este apelată, în momentul în care sunt declanșate întreruperi pe pinii 8, 9, 10 și 11. Teoretic, un radio ar trebui să furnizeze, pentru primele 4 canale, valori cuprinse între 1000 și 2000 de milisecunde, fapt ce nu se întâmplă (chiar și pentru cele mai performante telecomenzi RC). Astfel, este stabilită o funcție afină care duce intervalul de valori primit prin telecomandă, pentru fiecare dintre cele 4 canale, în intervalul [1000, 2000]. Acest proces, ce calibrează canalele, este prezentat în comentarii, în codul de mai jos. Apoi, programul citește datele de la modulul giroscop și accelerometru, le convertește și prelucrează corespunzător, calculând corecțiile ce trebuie aplicate esc-urilor, cu ajutorul algoritmilor PID. Evident, aceste corecții sunt influențate de comenzile primite de la telecomanda RC.

Un alt proces foarte important este calcularea IMU-ului (Inertial Measurement Unit), prin combinarea valorilor provenite de la modulul giroscop și accelerometru. IMU-ul este esențial pentru construcția unei drone, oferind informații despre orientare și mișcarea sa în spațiu, astfel încât aparatul de zbor să-și mențină poziția dorită. Accelerometrul măsoară accelerația liniară, ce permite IMU-ului să determine orientarea dronei relativă la forța gravitațională. Aceasta informație este foarte utilă când drona este statică sau se mișcă cu o viteză foarte mică. În schimb, accelerometrul poate fi afectat de forțe externe, precum vântul, vibrația și alte turbulențe. Pentru rezolvarea problemei, apare giroscopul. Giroscopul măsoară viteza unghiulară, ce permite IMU-ului să determine rata de rotație a

dronei. În principal, această caracteristică este utilă, când există o mișcare foarte rapidă cu manevre complexe. Totuși, giroscopul poate acumula erori în timp, proces cunoscut sub numele de "drift". Pentru a combina datele de la accelerometru și giroscop, cu scopul de a obține măsurători precise ale unui unghi, se utilizează o tehnică de fuziune a senzorilor. Astfel, cele două se completează reciproc și crează un IMU cu date mai precise și stabile, pentru orientarea și mișcarea în spațiu.

Odată ce se pot prelua și prelucra datele necesare, programul calculează corecțiile ce vor fi aplicate, astfel încât, drona să-și mențină poziția sa de echilibru. Calculele se fac cu ajutorul algoritmului PID care, după un proces de optimizare manuală, folosește nouă parametri aleși (câte trei pe fiecare mișcare: roll, pitch și yaw). La fiecare iterație, se aleg trei valori ideale corespunzătoare, cu ajutorul semnalelor receiverului. Cu cât eroare calculată va fi mai mică, cu atât unghiul calculat anterior va corespunde cu comanda dată prin telecomandă. Dacă acesta nu corespunde, eroarea va fi mai mare iar algoritmul PID va trebui să calculeze corecțiile necesare, pentru a minimiza-o, aducând drona în poziția dorită. Acest proces va fi repetat continuu și va începe doar prin anumite comenzi: pentru a porni motoarele, throttle-ul va fi în poziție inferioară și yaw în partea stângă și înapoi central, iar pentru a o opri, yaw va trebui să fie în partea dreaptă. Toate aceste procese, sunt explicate cu comentarii, în codurile sursă de mai jos

```
// includem biblioteca Wire.h pentru a comunica cu modulul giroscop prin protocolul I2C
#include <Wire.h>
// includem biblioteca EEPROM.h pentru a memora informatii in biblioteca EEPROM
#include <EEPROM.h>
// includem biblioteca math.h pentru a folosi functii specifice in procesul de calibrare
#include <math.h>
// declararea variabilelor globale pentru controler
// parametrii pentru roll
float pid_p_roll = 1.3;// parametrul P pentru miscarea roll
float pid_i_roll = 0.04;// parametrul I pentru miscarea roll
float pid_d_roll = 18.0;// parametrul D pentru miscarea roll
int pid_max_roll = 400;// maximul valorii de iesire a controlerului
// parametrii pentru pitch
float pid_p_pitch = 1.3;// parametrul P pentru miscarea pitch
float pid_i_pitch = 0.04;// parametrul I pentru miscarea pitch
float pid_d_pitch = 18.0;// parametrul D pentru miscarea pitch
int pid_max_pitch = 400;// maximul valorii de iesire a controlerului
// parametrii pentru yaw
float pid_p_yaw = 4.0;// parametrul P pentru miscarea yaw
float pid_i_gain_yaw = 0.02;// parametrul I pentru miscarea yaw
float pid_d_yaw = 0.0;// parametrul D pentru miscarea yaw
int pid_max_yaw = 400;// maximul valorii de iesire a controlerului
// declararea variabilelor globale
byte ultimul_canal_1, ultimul_canal_2, ultimul_canal_3, ultimul_canal_4;//calculul valorilor de la receiver
byte vector_date_eeprom[33]; // vectorul de valori, stocate anterior in procesul de configurare
byte highByte, lowByte;// byte-ul cel mai semnificativ si cel mai nesemnificativ
// variabile care vor stoca valorile provenite de la receiver
volatile int receiver_canal_1, receiver_canal_2, receiver_canal_3, receiver_canal_4;// valorile venite de la
receiverului
int esc_1, esc_2, esc_3, esc_4, throttle;// variabile pentru fiecare esc si throttle
int itere, start, adresa_giroscop;
int receiver_input[5];
int temperatura;
int acc_axis[4], gyro_axis[4];
float roll_level_adjust, pitch_level_adjust;
long acc_x, acc_y, acc_z, acc_total_vector;
```

```

unsigned long timer_canal_1, timer_canal_2, timer_canal_3, timer_canal_4, esc_timer, esc_loop_timer;
unsigned long timp_1, timp_2, timp_3, timp_4, timp_curent_intreruperi;
unsigned long loop_timer;
double gyro_pitch, gyro_roll, gyro_yaw;
double giro_ata_dif[4], giro_ata_med[4], giro_ata_val[4][2000];
float pid_eroare_temporara;
float pid_i_mem_roll, pid_roll_val_ideal, giro_roll_intrare, pid_iesire_roll, pid_ultima_eroare_d_roll;
float pid_i_mem_pitch, pid_pitch_val_ideal, giro_pitch_intrare, pid_iesire_pitch, pid_ultima_eroare_d_pitch;
float pid_i_mem_yaw, pid_yaw_val_ideal, giro_yaw_intrare, pid_iesire_yaw, pid_ultima_eroare_d_yaw;
float unghi_roll_acc, unghi_pitch_acc, unghi_pitch, unghi_roll;
boolean unghiurile_girosopului_setate_flag;
void setup(){
    // completam datele din eeprom intr-un vector ce au fost retinute la pasul de configurare
    // la fel si pentru adresa giroscopului
    copie_eeprom(); // facem asta pentru a avea acces mai usor la date
    Wire.begin(); // incepem comunicarea prin I2C, cu microcontroler-ul ca fiind Master
    // setam viteza de comunicare a I2C-ului la 400 kHz (modul rapid)
    // ca setare initiala, aceasta viteza este de 100kHz (modul minim)
    Wire.setClock(400000); // analog, se poate seta si prin schimbarea registrului TWBR = 12
    // Arduino (Atmega) are toti pinii declarati ca intrari
    // in schimb, trebuie sa declaram pinii pe care-i vrem ca iesiri
    pinMode(12, OUTPUT); // analog prin schimbarea valorii registrului DDRB |= B00110000
    pinMode(13, OUTPUT);
    // analog prin schimbarea valorii registrului DDRD |= B11110000, pentru fiecare
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);
    pinMode(6, OUTPUT);
    pinMode(7, OUTPUT);
    configurare_girosop(); // functie care configureaza registrii modulul giroscop si accelerometru
    // senzorul giroscop, ca si orice alt senzor, are o anumita acuratete in citirea datelor
    // iar datele furnizate reprezinta doar o aproximare a valorilor exacte
    // din acest motiv, pentru a micsora eroarea
    // este necesara calibrarea senzorului inainte de utilizarea datelor
    calibrare_girosop(); // vom citi mai multe date de la giroscop si vom face calibrarea sa
    // registrul PCICR controleaza 3 porturi de pini care pot declansa intreruperi
    // aceste 3 porturi/grupuri sunt controlate de alti 3 registri (PCMSK0, PCMSK1, PCMSK2)
    PCICR |= B00000001; // setam PCICR pentru a activa registrul PCMSK0, care controleaza pinii 8,9,10 si
    11
    PCMSK0 |= B00000001; // setam registrul PCINT0 (pinul 8) sa declanseze o intrerupere
    PCMSK0 |= B00000010; // setam registrul PCINT1 (pinul 9) sa declanseze o intrerupere
    PCMSK0 |= B00000100; // setam registrul PCINT2 (pinul 10) sa declanseze o intrerupere
    PCMSK0 |= B00010000; // setam registrul PCINT3 (pinul 11) sa declanseze o intrerupere
    // Wait until the receiver is active and the throttle is set to the lower position.
    // asteptam pana cand receiver-ul este activ si throttle-ul si yaw sunt in jos (pozitia sa minima)
    while(receiver_canal_3 < 990 || receiver_canal_3 > 1020 || receiver_canal_4 < 1400){
        receiver_canal_3 = convertire_semnal_receiver(3); // convertim semnalul efectiv al receiverului pentru
        throttle la standardul de 1000-2000us
        receiver_canal_4 = convertire_semnal_receiver(4); // convertim semnalul efectiv al receiverului pentru yaw
        la standardul de 1000-2000us
        // cat timp, esc-urile nu vor primi date sau comenzi, acestea vor bipai continuu
        // pentru ca nu vrem asta, le dam o valoare de 1000us pana cand sunt primite valorile de la receiver
        PORTD |= B11110000; // setam porturile 4, 5, 6, 7 ca high (cu tensiune electrica)
        delayMicroseconds(1000); // asteptam 1000us
        PORTD &= B00001111; // setam porturile 4, 5, 6, 7 ca low (fara tensiune electrica)
        delay(3); // asteptam 3 milisecunde pana la urmatoarea iteratie
    }
    start = 0; // setam variabila ce ne va indica startul pentru motoarele dronei, cu 0
}
// incepem programul principal, o functie care se va tot repeta
// aici vor avea loc calculele pe care algoritmiul PID ii vor face si vor comanda motoarele
void loop(){
    // in manualul giroscopului, 65.5 LBS = 1 grad/secunda, aceasta fiind metoda prin care datele digitale
    sunt convertite
    // cu o regula de trei simpla, rezulta ca gyro_data/65.5 = gyro_data in grade pe secunda

```



```

// pentru a filtra mai bine datele, acestea sunt proportionate
giro_roll_intrare = (giro_roll_intrare * 0.7) + ((gyro_roll / 65.5) * 0.3); // valoare in grade pe secunda
giro_pitch_intrare = (giro_pitch_intrare * 0.7) + ((gyro_pitch / 65.5) * 0.3); // valoare in grade pe secunda
giro_yaw_intrare = (giro_yaw_intrare * 0.7) + ((gyro_yaw / 65.5) * 0.3); // valoare in grade pe secunda
// stim ca viteza_unghiulara=distanța/timp. deci pentru a afla marimea unghiului parcurs,
// formula va fi, distanța=viteza_unghiulara*timp
// viteza_unghiulara este data de iesire a giroscopului iar timpul este data de frecventa (250 hz
// =0.004 secunde)
// 0.0000611 = 1 / 65.5 * 0.004
unghi_pitch += gyro_pitch * 0.0000611; // calculam unghiul parcurs prin miscarea pitch, si o adunam la
    unghi_pitch
unghi_roll += gyro_roll * 0.0000611; // calculam unghiul parcurs prin miscarea roll, si o adunam la
    unghi_pitch
// in momentul in care este data o comanda yaw, drona se roteste, ceea ce implica un transfer al
    unghiurilor
// pentru a putea pastra pozitia dronei corespunzatoare, in timpul zborului
// functia care descrie cel mai bine transformările pe care le face roll-ul sau pitch-ul este functia
    sinus
// functia sinus in Arduino, poate primi doar valorile in radiani, deci va trebui sa convertim din
    grade in radiani
// 0.000001066 = 0.0000611 * (3.142(PI) / 180grade)
unghi_pitch -= unghi_roll * sin(gyro_yaw * 0.000001066); // daca miscarea yaw transfera unghiul format
    prin roll in pitch
unghi_roll += unghi_pitch * sin(gyro_yaw * 0.000001066); // daca miscarea yaw transfera unghiul format
    prin pitch in rolls
// giroscopul este mai putin sensibil la vibratii si, in timp, datele acestuia intarzie
// astfel, pentru o buna performanta, trebuie sa ne folosim de datele de la giroscop si cele de la
    accelerometru
// va trebui sa calculam unghiul obtinut din vectorii dati de accelerometru
acc_total_vector = sqrt((acc_x*acc_x)+(acc_y*acc_y)+(acc_z*acc_z)); // calculam vectorul total al
    accelerometrului
unghi_pitch_acc = asin((float)acc_y/acc_total_vector)* 57.296; // calculam unghiul pitch
unghi_roll_acc = asin((float)acc_x/acc_total_vector)* -57.296; // calculam unghiul roll
// urmatoarele doua linii sunt folosite pentru calibrarea accelerometrului
unghi_pitch_acc -= 0.0; // calibrarea accelerometrului pentru valorile pitch
unghi_roll_acc -= 0.0; // calibrarea accelerometrului pentru valorile roll
// corectam problemele pe care le are giroscopul cu valorile accelerometrului
unghi_pitch = unghi_pitch * 0.9996 + unghi_pitch_acc * 0.0004;
unghi_roll = unghi_roll * 0.9996 + unghi_roll_acc * 0.0004;
// pentru a ajusta pozitia dronei, in timpul zborului, de folosim de urmatoarele doua variabile
// 1 grad al unghiului va fi corectat cu 15 pulsatii
pitch_level_adjust = unghi_pitch * 15; // calculam corectiile ce vor fi aplicate pentru unghiul pitch
roll_level_adjust = unghi_roll * 15; // calculam corectiile ce vor fi aplicate pentru unghiul roll
// pentru a porni motoarele, vom duce throttle-ul in pozitie inferioara si vom face yaw catre stanga
if(reciver_canal_3 < 1050 && reciver_canal_4 < 1050)
start = 1; // initializam variabila de start cu 1
// cand yaw este inapoi in pozitie centrala, pornim motoarele
if(start == 1 && reciver_canal_3 < 1050 && reciver_canal_4 > 1450){
    start = 2; // initializam variabila de start cu 2
    // in prima faza, pornim algoritmul cu valorile de initiale ale accelerometrului
    unghi_pitch = unghi_pitch_acc; // egalam ughiul pitch cu cel calculat anterior cu datele
        accelerometrului
    unghi_roll = unghi_roll_acc; // egalam ughiul roll cu cel calculat anterior cu datele accelerometrului
    unghiurile_giroscopului_setate_flag = true; // ne dorim sa facem acest lucru o singura data, deci
        initializam o variabila pentru asta
    // de fiecare data cand oprim si repornim motoarele, resetam variabilele ce ne vor ajuta pentru
        algoritmi PID
    pid_i_mem_roll = 0; // in aceasta variabila se retin valorile in procesul de integrare pentru
        algoritmi PID, pentru roll
    pid_ultima_eroare_d_roll = 0; // memoram ultima eroare folosita pentru procesul de derivare, pentru
        roll
    pid_i_mem_pitch = 0; // in aceasta variabila se retin valorile in procesul de integrare pentru
        algoritmi PID, pentru pitch
    pid_ultima_eroare_d_pitch = 0; // memoram ultima eroare folosita pentru procesul de derivare, pentru
        pitch

```

```

pid_i_mem_yaw = 0; // in aceasta variabila se retin valorile in procesul de integrare pentru
    algoritmi PID, pentru yaw
pid_ultima_eroare_d_yaw = 0; // memoram ultima eroare folosita pentru procesul de derivare, pentru
    yaw
}
// pentru a opri motoarele vom duce throttle-ul in pozitie inferioara si yaw in dreapta
if(start == 2 && reciver_canal_3 < 1050 && reciver_canal_4 > 1950)
start = 0; // initializam variabila de start cu 0
// variabila dorita este in grade pe secunda si este determinata de datele receiverului pentru miscarea
    roll
// pentru a face conversia din datele provenite de la reciver in grade, se divide valoarea la 3
pid_roll_val_ideala = 0;
// pentru mai buna functionaliata, se foloseste o "banda moarta" (o portiune in care valorile
    receiverului sunt 0)
// astfel, tanzitia dintr-o parte in alta sa fie mult mai lina
if(reciver_canal_1 > 1508) pid_roll_val_ideala = reciver_canal_1 - 1508;
else if(reciver_canal_1 < 1492) pid_roll_val_ideala = reciver_canal_1 - 1492;
pid_roll_val_ideala -= roll_level_adjust; // substragem corectiile aplicate unghiului din valoarea
    standard a roll-ului
pid_roll_val_ideala /= 3.0; // convertim valoarea ideala in grade, prin divizarea cu 3
// la fel procedam si pentru pitch an yaw
pid_pitch_val_ideala = 0; // pornim cu valoarea ideala egala cu 0
if(reciver_canal_2 > 1508) pid_pitch_val_ideala = reciver_canal_2 - 1508;
else if(reciver_canal_2 < 1492) pid_pitch_val_ideala = reciver_canal_2 - 1492;
pid_pitch_val_ideala -= pitch_level_adjust; // substragem corectiile aplicate unghiului din valoarea
    standard a pitch-ului
pid_pitch_val_ideala /= 3.0; // convertim valoarea ideala in grade, prin divizarea cu 3
pid_yaw_val_ideala = 0; // pornim cu valoarea ideala egala cu 0
if(reciver_canal_3 > 1050){ // facem o verificare, astfel incat, sa nu oprim motoarele
    if(reciver_canal_4 > 1508) pid_yaw_val_ideala = (reciver_canal_4 - 1508)/3.0;
    else if(reciver_canal_4 < 1492) pid_yaw_val_ideala = (reciver_canal_4 - 1492)/3.0;
}
calculare_pid(); // acum ca datele de intrare pentru algoritmul PID sunt cunoscute, calculam iesirile
throttle = reciver_canal_3; // pentru a coordona esc-urile, ne folosim de semnalul throttle-ului ca semnal
    de baza
if (start == 2){ // verificam daca motoarele sunt pornite
    // limitam cu o valoare throttle-ul.
    if (throttle > 1800) throttle = 1800; // in functie de rezultatele algoritmilor, se adauga sau nu
        diferenta de 2000 milisecunde
    esc_1 = throttle - pid_iesire_pitch + pid_iesire_roll - pid_iesire_yaw; // calculam valorile pentru esc-ul
        1 (fata dreapta)
    esc_2 = throttle + pid_iesire_pitch + pid_iesire_roll + pid_iesire_yaw; // calculam valorile pentru esc-ul
        2 (spate dreapta)
    esc_3 = throttle + pid_iesire_pitch - pid_iesire_roll - pid_iesire_yaw; // calculam valorile pentru esc-ul
        3 (spate stanga)
    esc_4 = throttle - pid_iesire_pitch - pid_iesire_roll + pid_iesire_yaw; // calculam valorile pentru esc-ul
        4 (fata stanga)
    // in functie de valorile obtinute, compensam in mod egal fiecare esc
    if (esc_1 < 1100) esc_1 = 1100; // continuam sa rotim motoarele
    if (esc_2 < 1100) esc_2 = 1100; // continuam sa rotim motoarele
    if (esc_3 < 1100) esc_3 = 1100; // continuam sa rotim motoarele
    if (esc_4 < 1100) esc_4 = 1100; // continuam sa rotim motoarele
    if (esc_1 > 2000) esc_1 = 2000; // limitam esc-urile la 2000 de milisecunde
    if (esc_2 > 2000) esc_2 = 2000; // limitam esc-urile la 2000 de milisecunde
    if (esc_3 > 2000) esc_3 = 2000; // limitam esc-urile la 2000 de milisecunde
    if (esc_4 > 2000) esc_4 = 2000; // limitam esc-urile la 2000 de milisecunde
}
else{ // caz in care, startul nu este 2 si vrem totusi sa tinem motoarele in miscare
    esc_1 = 1000;
    esc_2 = 1000;
    esc_3 = 1000;
    esc_4 = 1000;
}
// rata de improspatare este de 250 hz, deci va trebui sa schimbam datele pentru escuri la fiecare
    0,004 secunde

```

```

while(micros() - loop_timer < 4000); // asteptam pana cand cele 0.004 secunde au trecut
loop_timer = micros(); // setam timpul pentru urmatoarea iteratie
PORTD |= B11110000; // punem sub tensiune pinii 4,5,6 si 7
timer_canal_1 = esc_1 + loop_timer; // pentru fiecare canal, calculam timpul corespunzator esc-ului
timer_canal_2 = esc_2 + loop_timer; // pentru fiecare canal, calculam timpul corespunzator esc-ului
timer_canal_3 = esc_3 + loop_timer; // pentru fiecare canal, calculam timpul corespunzator esc-ului
timer_canal_4 = esc_4 + loop_timer; // pentru fiecare canal, calculam timpul corespunzator esc-ului
semnale_giroscopec(); // preluam noile date de la giroscop si convertim in grade cele de la reciver
while(PORTD >= 16){ // stam in bucla repetitiva, pana cand pinii setati anterior nu mai sunt sub
    tensiune
    esc_loop_timer = micros(); // citim timpul real
    // cand depasim limita de timp, putem sa oprim tensiunea pentru pinii corespunzatori esc-urilor
    if(timer_canal_1 <= esc_loop_timer) PORTD &= B11101111; // eliminam tensiunea pentru pinul 4
    if(timer_canal_2 <= esc_loop_timer) PORTD &= B11011111; // eliminam tensiunea pentru pinul 5
    if(timer_canal_3 <= esc_loop_timer) PORTD &= B10111111; // eliminam tensiunea pentru pinul 6
    if(timer_canal_4 <= esc_loop_timer) PORTD &= B01111111; // eliminam tensiunea pentru pinul 7
}
}
// aceasta functie este apelata de fiecare data cand valorile de intrare de la pinii 8,9,10 sau 11 se
// schimba
ISR(PCINT0_vect){ // rutina intreruperilor
    timp_curent_intreruperi = micros(); //memoram timpul curent al microcontroler-ului
    // PINB este registrul care controleaza portul piniilor de la 8 la 13
    // verificam de unde a fost declansata o intrerupere
    // canal 1
    if(PINB & B00000001){ // verificam daca avem intrerupere pe pinul 8
        // variabila care ne spune ca a inceput intreruperea
        if(ultimul_canal_1 == 0){ // verificam daca nu exista intrerupere
            ultimul_canal_1 = 1; // schimbam valoarea spunand ca avem intrerupere
            timp_1 = timp_curent_intreruperi; //setam timp_1 cu timp_curent_intreruperi
        }
    }
    else if(ultimul_canal_1 == 1){ // verificam daca exista intrerupere
        ultimul_canal_1 = 0; // schimbam valoare spunand ca nu mai avem intrerupere
        reciver_canal_1 = timp_curent_intreruperi - timp_1; // valoare canalului va fi egala cu
            timp_curent_intreruperi - timp_1
    }
    // canal 2
    if(PINB & B00000010){ // verificam daca avem intrerupere pe pinul 9
        if(ultimul_canal_2 == 0){ // verificam daca nu exista intrerupere
            ultimul_canal_2 = 1; // schimbam valoarea spunand ca avem intrerupere
            timp_2 = timp_curent_intreruperi; //setam timp_2 cu timp_curent_intreruperi
        }
    }
    else if(ultimul_canal_2 == 1){ // verificam daca exista intrerupere
        ultimul_canal_2 = 0; // schimbam valoare spunand ca nu mai avem intrerupere
        reciver_canal_2 = timp_curent_intreruperi - timp_2; // valoare canalului va fi egala cu
            timp_curent_intreruperi - timp_2
    }
    // canal 3
    if(PINB & B00000100){ // verificam daca avem intrerupere pe pinul 10
        if(ultimul_canal_3 == 0){ // verificam daca nu exista intrerupere
            ultimul_canal_3 = 1; // schimbam valoarea spunand ca avem intrerupere
            timp_3 = timp_curent_intreruperi; //setam timp_3 cu timp_curent_intreruperi
        }
    }
    else if(ultimul_canal_3 == 1){ // verificam daca exista intrerupere
        ultimul_canal_3 = 0; // schimbam valoare spunand ca nu mai avem intrerupere
        reciver_canal_3 = timp_curent_intreruperi - timp_3; // valoare canalului va fi egala cu
            timp_curent_intreruperi - timp_3
    }
    // canal 4
    if(PINB & B00001000){ // verificam daca avem intrerupere pe pinul 11
        if(ultimul_canal_4 == 0){ // verificam daca nu exista intrerupere

```

```

        ultimul_canal_4 = 1; // schimbam valoarea spunand ca avem intrerupere
        timp_4 = timp_curent_intreruperi; //setam timp_4 cu timp_curent_intreruperi
    }
}
else if(ultimul_canal_4 == 1){ // verificam daca exista intrerupere
    ultimul_canal_4 = 0; // schimbam valoarea spunand ca nu mai avem intrerupere
    reciver_canal_4 = timp_curent_intreruperi - timp_4; // valoare canalului va fi egala cu
        timp_curent_intreruperi - timp_4
    }
}
// functie care citeste de la giroscop si accelerometru date
void semnale_giroscop(){
    Wire.beginTransmission(adresa_giroscop); // incepem comunicarea cu modulul giroscop si accelerometru
    // registrul 3B retine date pentru masuratorile accelerometrului
    Wire.write(0x3B); // anuntam ca vrem sa citim, incepand cu adresa 3B (59, Accelerometer Measurements)
    Wire.endTransmission(); // opresc transmiterea de date
    Wire.requestFrom(adresa_giroscop, 14); // cerem de la adresa respectiva 14 bytes
    reciver_canal_1 = convertire_semnal_reciver(1); //convertim val primita de la reciver pentru pitch la o val
        standard de 1000 - 2000us
    reciver_canal_2 = convertire_semnal_reciver(2); //convertim val primita de la reciver pentru roll la o val
        standard de 1000 - 2000us
    reciver_canal_3 = convertire_semnal_reciver(3); //convertim val primita de la reciver pentru throttle la o
        val standard de 1000 - 2000us
    reciver_canal_4 = convertire_semnal_reciver(4); //convertim val primita de la reciver pentru yaw la o val
        standard de 1000 - 2000us
    // pe masura ce citim, functia va face un auto-increment astfel incat, sa citim pana la registrul 72 (
        Gyroscope Measurements)
    while(Wire.available() < 14); // asteptam pana cand cei 14 bytes sunt primiti
    acc_axis[1] = Wire.read() << 8 | Wire.read(); // memoram byte-ul cel mai ne semnificativ si cel mai
        semnificativ al variabilei acc_x
    acc_axis[2] = Wire.read() << 8 | Wire.read(); // memoram byte-ul cel mai ne semnificativ si cel mai
        semnificativ al variabilei acc_y
    acc_axis[3] = Wire.read() << 8 | Wire.read(); // memoram byte-ul cel mai ne semnificativ si cel mai
        semnificativ al variabilei acc_z
    temperatura = Wire.read() << 8 | Wire.read(); // memoram byte-ul cel mai ne semnificativ si cel mai
        semnificativ al variabilei temperatura
    gyro_axis[1] = Wire.read() << 8 | Wire.read(); // memoram byte-ul cel mai ne semnificativ si cel mai
        semnificativ al valorilor unghiurilor
    gyro_axis[2] = Wire.read() << 8 | Wire.read(); // memoram byte-ul cel mai ne semnificativ si cel mai
        semnificativ al valorilor unghiurilor
    gyro_axis[3] = Wire.read() << 8 | Wire.read(); // memoram byte-ul cel mai ne semnificativ si cel mai
        semnificativ al valorilor unghiurilor
    // verificam daca a fost facuta calibrarea
    if(iterare == 2000){ // daca a fost facuta
        gyro_axis[1] -= giro_axa_diff[1]; // compensam cu valorile obtinute la calibrare
        gyro_axis[2] -= giro_axa_diff[2]; // compensam cu valorile obtinute la calibrare
        gyro_axis[3] -= giro_axa_diff[3]; // compensam cu valorile obtinute la calibrare
    }
    // setam valorile corespunzatoare, pentru fiecare tip de date
    // pentru datele de la giroscop si accelerometru
    gyro_roll = gyro_axis[vector_date_eeprom[28] & 0b00000011];
    if(vector_date_eeprom[28] & 0b10000000) gyro_roll *= -1;
    gyro_pitch = gyro_axis[vector_date_eeprom[29] & 0b00000011];
    if(vector_date_eeprom[29] & 0b10000000) gyro_pitch *= -1;
    gyro_yaw = gyro_axis[vector_date_eeprom[30] & 0b00000011];
    if(vector_date_eeprom[30] & 0b10000000) gyro_yaw *= -1;
    acc_x = acc_axis[vector_date_eeprom[29] & 0b00000011];
    if(vector_date_eeprom[29] & 0b10000000) acc_x *= -1;
    acc_y = acc_axis[vector_date_eeprom[28] & 0b00000011];
    if(vector_date_eeprom[28] & 0b10000000) acc_y *= -1;
    acc_z = acc_axis[vector_date_eeprom[30] & 0b00000011];
    if(vector_date_eeprom[30] & 0b10000000) acc_z *= -1;
}
// functie care calculeaza valorile de iesire ai algoritmilor PID
void calculare_pid(){

```

```

//Calulcul pentru Roll
pid_eroare_temporara = pid_roll_val_ideala - giro_roll_intrare;// calculam eroarea
pid_i_mem_roll += pid_i_roll * pid_eroare_temporara;// cu ajutorul parametrului de integrare, integram
eroarea
if(pid_i_mem_roll > pid_max_roll)pid_i_mem_roll = pid_max_roll;// ne propunem o valoare maxima, pe
care nu vrem sa o depasim
else if(pid_i_mem_roll < pid_max_roll * -1)pid_i_mem_roll = pid_max_roll * -1;
// calculam valoarea de iesire a controlerului, corespunzator formulei prezentate anterior
pid_iesire_roll = pid_p_roll * pid_eroare_temporara + pid_i_mem_roll + pid_d_roll * (pid_eroare_temporara
- pid_ultima_eroare_d_roll);
if(pid_iesire_roll > pid_max_roll)pid_iesire_roll = pid_max_roll; // verificam daca a fost depasita
valoarea maxima
else if(pid_iesire_roll < pid_max_roll * -1)pid_iesire_roll = pid_max_roll * -1;
// retinem ultima valoare, care ne va ajuta in calculul controlerului de derivabilitate
pid_ultima_eroare_d_roll = pid_eroare_temporara;
//Calulcul pentru Pitch
pid_eroare_temporara = pid_pitch_val_ideala - giro_pitch_intrare;// calculam eroarea
pid_i_mem_pitch += pid_i_pitch * pid_eroare_temporara;// cu ajutorul parametrului de integrare,
integram eroarea
if(pid_i_mem_pitch > pid_max_pitch)pid_i_mem_pitch = pid_max_pitch;// ne propunem o valoare maxima
, pe care nu vrem sa o depasim
else if(pid_i_mem_pitch < pid_max_pitch * -1)pid_i_mem_pitch = pid_max_pitch * -1;
// calculam valoarea de iesire a controlerului, corespunzator formulei prezentate anterior
pid_iesire_pitch = pid_p_pitch * pid_eroare_temporara + pid_i_mem_pitch + pid_d_pitch * (
pid_eroare_temporara - pid_ultima_eroare_d_pitch);
if(pid_iesire_pitch > pid_max_pitch)pid_iesire_pitch = pid_max_pitch; // verificam daca a fost depasita
valoarea maxima
else if(pid_iesire_pitch < pid_max_pitch * -1)pid_iesire_pitch = pid_max_pitch * -1;
// retinem ultima valoare, care ne va ajuta in calculul controlerului de derivabilitate
pid_ultima_eroare_d_pitch = pid_eroare_temporara;
//Calulcul pentru Yaw
pid_eroare_temporara = pid_yaw_val_ideala - giro_yaw_intrare;// calculam eroarea
pid_i_mem_yaw += pid_i_gain_yaw * pid_eroare_temporara;// cu ajutorul parametrului de integrare,
integram eroarea
if(pid_i_mem_yaw > pid_max_yaw)pid_i_mem_yaw = pid_max_yaw;// ne propunem o valoare maxima, pe
care nu vrem sa o depasim
else if(pid_i_mem_yaw < pid_max_yaw * -1)pid_i_mem_yaw = pid_max_yaw * -1;
// calculam valoarea de iesire a controlerului, corespunzator formulei prezentate anterior
pid_iesire_yaw = pid_p_yaw * pid_eroare_temporara + pid_i_mem_yaw + pid_d_yaw * (
pid_eroare_temporara - pid_ultima_eroare_d_yaw);
if(pid_iesire_yaw > pid_max_yaw)pid_iesire_yaw = pid_max_yaw; // verificam daca a fost depasita
valoarea maxima
else if(pid_iesire_yaw < pid_max_yaw * -1)pid_iesire_yaw = pid_max_yaw * -1;
// retinem ultima valoare, care ne va ajuta in calculul controlerului de derivabilitate
pid_ultima_eroare_d_yaw = pid_eroare_temporara;
}
// aceasta functie converteste val primita de la reciver intr-o val standard de 1000-1500-2000 de
microsecunde
// inseamna stabilirea unei functii afine care sa duca intervalul de valori al telecomenzii
// pentru fiecare dintre cele 4 canale, in intervalul [1000, 2000]
// pentru asta ne folosim de valorile minime, maxime si centrale, memorate in EEPROM
int convertire_semnal_reciver(byte function){
byte canal, inversat;// declaram cateva variabile ajutatoare
int low, center, high, semnal_efectiv;
int diferenta;
// fiecarui canal ii corespunde o anumita functie, care defineste o anumita miscare
// spre exemplu, pentru canalul 3, este atribuita miscare throttle-ului, adica 1
// aceasta memorare este facuta, pentru a se pastra o ordine
canal = vector_date_eeprom[function + 23] & 0b00000111;// verificam din EEPROM, care miscare corespunde
cu functia
if(vector_date_eeprom[function + 23] & 0b10000000)inversat = 1; // caz in care canalele sunt inversate
else inversat = 0;
semnal_efectiv = receiver_input[canal]; // pentru ca stim canalul corespunzator, vom sti ce data de intrare
a reciverului sa alegem
// pentru fiecare canal, luam cea mai mica valoarea, cea mai mare si cea centrala

```

```

low = (vector_date_eeprom[canal * 2 + 15] << 8) | vector_date_eeprom[canal * 2 + 14]; //memoram valoarea
    minima pentru canalul specific
center = (vector_date_eeprom[canal * 2 - 1] << 8) | vector_date_eeprom[canal * 2 - 2]; //memoram valoarea
    centrala pentru canalul specific
high = (vector_date_eeprom[canal * 2 + 7] << 8) | vector_date_eeprom[canal * 2 + 6]; //memoram valoarea
    maxima ce pentru canalul specific
// pentru calculare, in functie de semnal, alegem functia corespunzatoare
// f(semnal_reciver) = 1500 + (semnal_reciver - semnal_central)/(semnal_central-semnal_minim)*500
if(semnal_efectiv < center){ // cand semnalul efectiv este mai mic decat cel central
    if(semnal_efectiv < low) semnal_efectiv = low; // limitam valoarea la cea care a fost detectata in timpul
        calibrarii
    diferenta = ((long)(center - semnal_efectiv) * (long)500) / (center - low);
    if(inversat == 1) return 1500 + diferenta; //daca canalul este inversat
    else return 1500 - diferenta; //daca canalul nu este inversat
} // f(semnal_reciver) = 1500 + (semnal_reciver - semnal_central)/(semnal_maxim-semnal_central)*500
else if(semnal_efectiv > center){ // cand semnalul efectiv este mai mare decat cel central
    if(semnal_efectiv > high) semnal_efectiv = high; // limitam valoarea la cea care a fost detectata in
        timpul calibrarii
    diferenta = ((long)(semnal_efectiv - center) * (long)500) / (high - center);
    if(inversat == 1) return 1500 - diferenta; //daca canalul este inversat
    else return 1500 + diferenta; //daca canalul nu este inversat
}
} else return 1500; // daca valoare corespunde cu cea centrala
}
// functie folosita pentru a configura modulul giroscop si accelerometru
void configurare_giroscop(){
    // incepem sa configuram modulul giroscop si accelerometru
    // pentru a seta anumite caracteristici ale datelor pe care le vom primi
    Wire.beginTransmission(adresa_giroscop); // incepem comunicarea cu giroscopul si accelerometru
    //in prima faza, giroscopul vine mod activat, sleep mode
    //acesta va trebui schimbat, pentru a putea porni giroscopul
    Wire.write(0x6B); // registrul 107 (PWR_MGMT_1) permite configurarea modulului de alimentare si sursa
        ceasului
    Wire.write(0x00); // schimbam valoarea registrului PWR_MGMT_1 la 0 pentru a porni giroscopul
    Wire.endTransmission(); // oprim transmiterea de date si trimitem registrul
    Wire.beginTransmission(adresa_giroscop); // incepem comunicarea cu giroscopul si accelerometru
    Wire.write(0x1B); // anuntam ca vrem sa scriem in registrul GYRO_CONFIG
    Wire.write(0x08); // setam bitii registrului cu 00001000 (1 pentru bitul FS_SEL=Full scale range)(500dps)
        - degree per second
    // astfel, selectam gama completa de valori ale giroscopului
    Wire.endTransmission(); // oprim transmiterea de date
    Wire.beginTransmission(adresa_giroscop); // incepem comunicarea cu giroscopul si accelerometru
    //0x1C sau registrul ACCEL_CONFIG este folosit pentru a seta intervalul pe care datele accelerometrului
        sa le aiba (+/- 2g/16g)
    Wire.write(0x1C); // vrem sa scriem la adresa 28 ( registrul ACCEL_CONFIG)
    Wire.write(0x10); // setam bitii registrului cu 00010000, pentru a stabili valoarea intervalului de date
        (+/- 8g full scale range)
    Wire.endTransmission(); // oprim transmiterea de date
    Wire.beginTransmission(adresa_giroscop); // incepem comunicarea cu giroscopul si accelerometru
    // prin natura sa, giroscopul este sensibil la vibratii, astfel, pentru performante mai bune putem
        configura un registru
    // registrul 1A sau 26 este folosit pentru configurare
    Wire.write(0x1A); // vrem sa scriem la adresa 26 (registrul CONFIG)
    Wire.write(0x03); // setam bitii registrului cu 00000011 DLPF (Digital Low Pass Filter) la aproximativ 43
        Hz
    Wire.endTransmission(); // oprim transmiterea de date
}
// functie care copiaza datele din eeprom intr-un vector
void copiere_eeprom(){
    for(start = 0; start <= 33; start++){
        vector_date_eeprom[start] = EEPROM.read(start);
        start = 0; // setam valoarea de start inapoi la 0
        adresa_giroscop = vector_date_eeprom[32]; // memoram adresa modulului giroscop si accelerometru
    }
}
void calibrare_giroscop(){

```

```

for (iterare = 0; iterare < 2000 ; iterare ++){ //luam 2000 de date de la giroscop.
    semnale_giroscop();// se citesc date de la giroscop si accelerometru
    giro_axa_med[1] += gyro_axis[1];// adaugam valoarea pentru roll corespunzatoare giroscopului
    giro_axa_val[1][iterare] = gyro_axis[1];
    giro_axa_med[2] += gyro_axis[2];//adaugam valoarea pentru pitch corespunzatoare giroscopului
    giro_axa_val[2][iterare] = gyro_axis[2];
    giro_axa_med[3] += gyro_axis[3];//adaugam valoarea pentru yaw corespunzatoare giroscopului
    giro_axa_val[3][iterare] = gyro_axis[3];
    // cat timp, esc-urile nu vor primi date sau comenzi, acestea vor bipai continuu
    // pentru ca nu vrem asta, le dam o valoare de 1000us si apoi le oprim
    PORTD |= B11110000;// setam porturile 4, 5, 6, 7 ca high (cu tensiune electrica)
    delayMicroseconds(1000);// asteptam 1000us
    PORTD &= B00001111;// setam porturile 4, 5, 6, 7 ca low (fara tensiune electrica)
    delay(3);// asteptam 3 milisecunde pana la urmatoarea iteratie
}
// dupa ce avem 2000 de masuratori, putem sa facem o medie cu acestea pentru a gasi "eroarea"
//giroscopului
giro_axa_med[1] /= 2000;// impartim la 2000 valorile pentru roll
giro_axa_med[2] /= 2000;// impartim la 2000 valorile pentru pitch
giro_axa_med[3] /= 2000;// impartim la 2000 valorile pentru yaw
for (int i = 0; i < 2000 ; i ++){ // adunam distanta drinte fiecare masuratore si media anterioara
    giro_axa_dif[1] += (giro_axa_val[1][i] - giro_axa_med[1]) * (giro_axa_val[1][i] - giro_axa_med[1]);
    giro_axa_dif[2] += (giro_axa_val[2][i] - giro_axa_med[2]) * (giro_axa_val[2][i] - giro_axa_med[2]);
    giro_axa_dif[3] += (giro_axa_val[3][i] - giro_axa_med[3]) * (giro_axa_val[3][i] - giro_axa_med[3]);
}
//calculam abaterea standard de la media obtinuta anterior
giro_axa_dif[1] /= 2000;
giro_axa_dif[2] /= 2000;
giro_axa_dif[3] /= 2000;
giro_axa_dif[1] = sqrt(giro_axa_dif[1]);
giro_axa_dif[2] = sqrt(giro_axa_dif[2]);
giro_axa_dif[3] = sqrt(giro_axa_dif[3]);
}

```

Bibliografie

- [1] Karl Johan Åström, Tore Hägglund, *PID Controllers: Theory, Design and Tuning*
- [2] Barry Davies, *Build a Drone: A Step-by-Step Guide to Designing, Constructing, and Flying Your Very Own Drone*
- [3] Joop Brokking, http://www.brokking.net/ymfc-al_main.html
- [4] Oscar Liang, <https://oscarliang.com/pid/>